

```

int ClosestInBST(struct BinaryTreeNode *root, int key){
    struct BinaryTreeNode *temp, *element;
    struct Queue *Q;
    int difference = INT_MAX;
    if(!root)
        return 0;
    Q = CreateQueue();
    EnQueue(Q, root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        if(difference > (abs(temp->data-key))){
            difference = abs(temp->data-key);
            element = temp;
        }
        if(temp->left)
            EnQueue (Q, temp->left);
        if(temp->right)
            EnQueue (Q, temp->right);
    }
    DeleteQueue(Q);
    return element->data;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-83 For [Problem-82](#), can we solve it using the recursive approach?

Solution: The approach is similar to [Problem-18](#). Following is a simple algorithm for finding the closest Value in BST.

1. If the root is NULL, then the closest value is zero (or NULL).
2. If the root's data matches the given key, then the closest is the root.
3. Else, consider the root as the closest and do the following:
 - a. If the key is smaller than the root data, find the closest on the left side tree of the root recursively and call it temp.
 - b. If the key is larger than the root data, find the closest on the right side tree of the root recursively and call it temp.
4. Return the root or temp depending on whichever is nearer to the given key.

```

struct BinaryTreeNode * ClosestInBST(struct BinaryTreeNode *root, int key){
    struct BinaryTreeNode *temp;
    if(root == NULL)
        return root;
    if(root->data == key)
        return root;
    if(key < root->data){
        if(!root->left)
            return root;
        temp = ClosestInBST(root->left, key);
        return abs(temp->data-key) > abs(root->data-key) ? root : temp;
    }else{
        if(!root->right)
            return root;
        temp = ClosestInBST(root->right, key);
        return abs(temp->data-key) > abs(root->data-key) ? root : temp;
    }
    return NULL;
}

```

Time Complexity: $O(n)$ in worst case, and in average case it is $O(\log n)$.

Space Complexity: $O(n)$ in worst case, and in average case it is $O(\log n)$.

Problem-84 Median in an infinite series of integers

Solution: Median is the middle number in a sorted list of numbers (if we have odd number of elements). If we have even number of elements, median is the average of two middle numbers in a sorted list of numbers.

For solving this problem we can use a binary search tree with additional information at each node, and the number of children on the left and right subtrees. We also keep the number of total nodes in the tree. Using this additional information we can find the median in $O(\log n)$ time, taking the appropriate branch in the tree based on the number of children on the left and right of the current node. But, the insertion complexity is $O(n)$ because a standard binary search tree can degenerate into a linked list if we happen to receive the numbers in sorted order.

So, let's use a balanced binary search tree to avoid worst case behavior of standard binary search trees. For this problem, the balance factor is the number of nodes in the left subtree minus the number of nodes in the right subtree. And only the nodes with a balance factor of ± 1 or 0 are considered to be balanced.

So, the number of nodes on the left subtree is either equal to or 1 more than the number of nodes on the right subtree, but not less.

If we ensure this balance factor on every node in the tree, then the root of the tree is the median, if the number of elements is odd. In the number of elements is even, the median is the average of the root and its inorder successor, which is the leftmost descendent of its right subtree.

So, the complexity of insertion maintaining a balanced condition is $O(\log n)$ and finding a median operation is $O(1)$ assuming we calculate the inorder successor of the root at every insertion if the number of nodes is even.

Insertion and balancing is very similar to AVL trees. Instead of updating the heights, we update the number of nodes information. Balanced binary search trees seem to be the most optimal solution, insertion is $O(\log n)$ and find median is $O(1)$.

Note: For an efficient algorithm refer to the [Priority Queues and Heaps](#) chapter.

Problem-85 Given a binary tree, how do you remove all the half nodes (which have only one child)? Note that we should not touch leaves.

Solution: By using post-order traversal we can solve this problem efficiently. We first process the left children, then the right children, and finally the node itself. So we form the new tree bottom up, starting from the leaves towards the root. By the time we process the current node, both its left and right subtrees have already been processed.

```
struct BinaryTreeNode *removeHalfNodes(struct BinaryTreeNode *root){
    if (!root)
        return NULL;
    root->left=removeHalfNodes(root->left);
    root->right=removeHalfNodes(root->right);
    if (root->left == NULL && root->right == NULL)
        return root;
    if (root->left == NULL)
        return root->right;
    if (root->right == NULL)
        return root->left;
    return root;
}
```

Time Complexity: $O(n)$.

Problem-86 Given a binary tree, how do you remove its leaves?

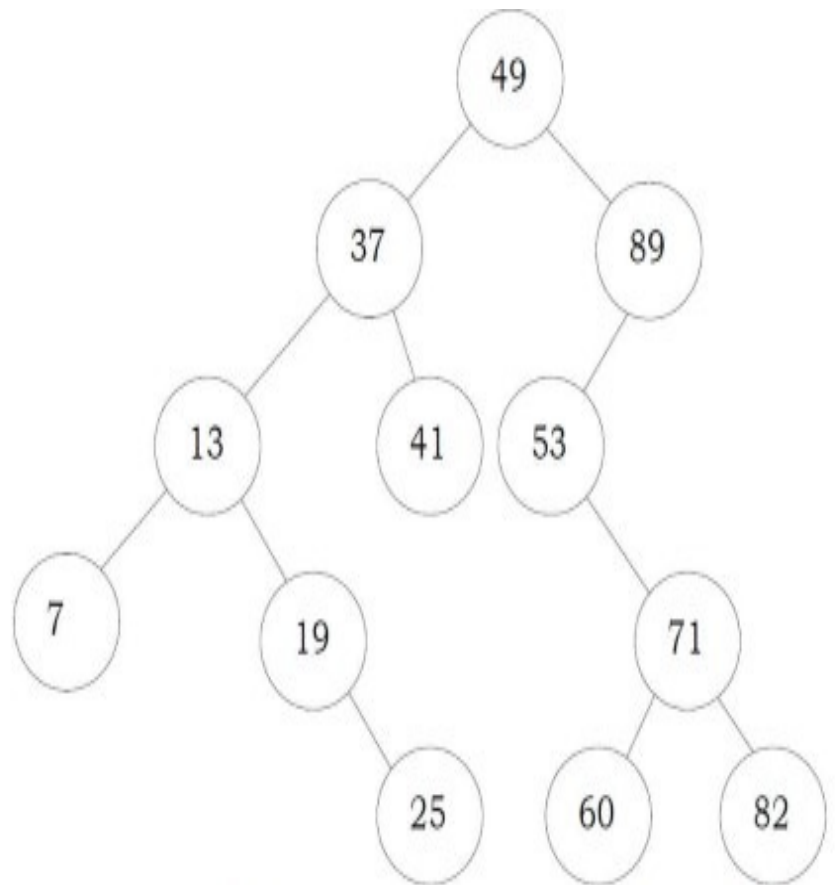
Solution: By using post-order traversal we can solve this problem (other traversals would also work).

```
struct BinaryTreeNode* removeLeaves(struct BinaryTreeNode* root) {  
    if (root != NULL) {  
        if (root->left == NULL && root->right == NULL) {  
            free(root);  
            return NULL;  
        } else {  
            root->left = removeLeaves(root->left);  
            root->right = removeLeaves(root->right);  
        }  
    }  
    return root;  
}
```

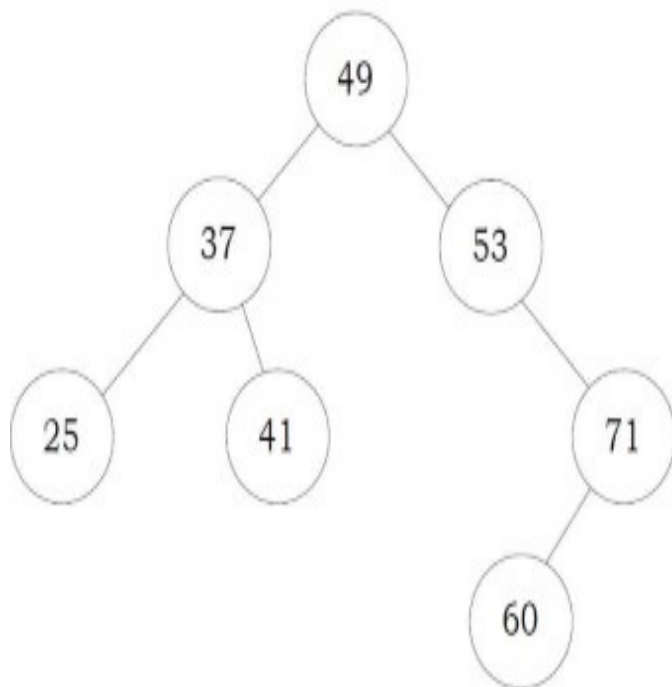
Time Complexity: $O(n)$.

Problem-87 Given a BST and two integers (minimum and maximum integers) as parameters, how do you remove (prune) elements that are not within that range?

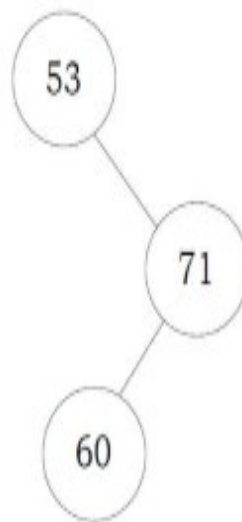
Sample Tree



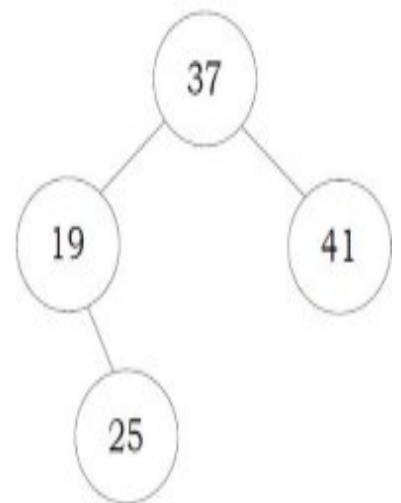
PruneBST(24,71);



PruneBST(53,79);



PruneBST(17,41);



Solution: Observation: Since we need to check each and every element in the tree, and the subtree changes should be reflected in the parent, we can think about using post order traversal. So we process the nodes starting from the leaves towards the root. As a result, while processing the node itself, both its left and right subtrees are valid pruned BSTs. At each node we will return

a pointer based on its value, which will then be assigned to its parent's left or right child pointer, depending on whether the current node is the left or right child of the parent. If the current node's value is between A and B ($A \leq \text{node's data} \leq B$) then no action needs to be taken, so we return the reference to the node itself.

If the current node's value is less than A , then we return the reference to its right subtree and discard the left subtree. Because if a node's value is less than A , then its left children are definitely less than A since this is a binary search tree. But its right children may or may not be less than A ; we can't be sure, so we return the reference to it. Since we're performing bottom-up post-order traversal, its right subtree is already a trimmed valid binary search tree (possibly NULL), and its left subtree is definitely NULL because those nodes were surely less than A and they were eliminated during the post-order traversal.

A similar situation occurs when the node's value is greater than B , so we now return the reference to its left subtree. Because if a node's value is greater than B , then its right children are definitely greater than B . But its left children may or may not be greater than B ; So we discard the right subtree and return the reference to the already valid left subtree.

```
struct BinarySearchTreeNode* PruneBST(struct BinarySearchTreeNode *root, int A, int B){
    if(!root) return NULL;
    root->left= PruneBST(root->left,A,B);
    root->right= PruneBST(root->right,A,B);
    if(A<=root->data && root->data<=B)
        return root;
    if(root->data<A)
        return root->right;
    if(root->data>B)
        return root->left;
}
```

Time Complexity: $O(n)$ in worst case and in average case it is $O(\log n)$.

Note: If the given BST is an AVL tree then $O(n)$ is the average time complexity.

Problem-88 Given a binary tree, how do you connect all the adjacent nodes at the same level? Assume that given binary tree has next pointer along with left and right pointers as shown below.

```
struct BinaryTreeNode {  
    int data;  
    struct BinaryTreeNode *left;  
    struct BinaryTreeNode *right;  
    struct BinaryTreeNode *next;  
};
```

Solution: One simple approach is to use level-order traversal and keep updating the next pointers. While traversing, we will link the nodes on the next level. If the node has left and right node, we will link left to right. If node has next node, then link rightmost child of current node to leftmost child of next node.


```

void linkingNodesOfSameLevel(struct BinaryTreeNode *root){
    struct Queue *Q = CreateQueue();
    struct BinaryTreeNode *prev;    // Pointer to the previous node of the current level
    struct BinaryTreeNode *temp;
    int currentLevelNodeCount, nextLevelNodeCount;
    if(!root)
        return;
    EnQueue(Q, root);
    currentLevelNodeCount = 1;
    nextLevelNodeCount = 0;
    prev = NULL;
    while (!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        if (temp->left){
            EnQueue(Q, temp->left);
            nextLevelNodeCount++;
        }
        if (temp->right){
            EnQueue(Q, temp->right);
            nextLevelNodeCount++;
        }
        // Link the previous node of the current level to this node
        if (prev)
            prev->next = temp;
        // Set the previous node to the current
        prev = temp;
        currentLevelNodeCount--;
        if (currentLevelNodeCount == 0) { // if this is the last node of the current level
            currentLevelNodeCount = nextLevelNodeCount;
            nextLevelNodeCount = 0;
            prev = NULL;
        }
    }
}

```


Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-89 Can we improve space complexity for [Problem-88](#)?

Solution: We can process the tree level by level, but without a queue. The logical part is that when we process the nodes of the next level, we make sure that the current level has already been linked.

```
void linkingNodesOfSameLevel(struct BinaryTreeNode *root) {
    if(!root) return;
    struct BinaryTreeNode *rightMostNode = NULL, *nextHead = NULL, *temp = root;
    //connect next level of current root node level
    while(temp!= NULL){
        if(temp->left!= NULL)
            if(rightMostNode== NULL){
                rightMostNode=temp->left;
                nextHead=temp->left;
            }
            else{
                rightMostNode->next = temp->left;
                rightMostNode = rightMostNode->next;
            }
        if(temp->right!= NULL)
            if(rightMostNode== NULL){
                rightMostNode=temp->right;
                nextHead=temp->right;
            }
            else{
                rightMostNode->next = temp->right;
                rightMostNode = rightMostNode->next;
            }
        temp=temp->next;
    }
    linkingNodesOfSameLevel(nextHead);
}
```

Time Complexity: $O(n)$. Space Complexity: $O(\text{depth of tree})$ for stack space.

Problem-90 Assume that a set S of n numbers are stored in some form of balanced binary search tree; i.e. the depth of the tree is $O(\log n)$. In addition to the key value and the pointers to children, assume that every node contains the number of nodes in its subtree. Specify a reason(s) why a balanced binary tree can be a better option than a complete binary tree for storing the set S .

Solution: Implementation of a balanced binary tree requires less RAM space as we do not need to keep the complete tree in RAM (since they use pointers).

Problem-91 For the [Problem-90](#), specify a reason (s) why a complete binary tree can be a better option than a balanced binary tree for storing the set S .

Solution: A complete binary tree is more space efficient as we do not need any extra flags. A balanced binary tree usually takes more space since we need to store some flags. For example, in a Red-Black tree we need to store a bit for the color. Also, a complete binary tree can be stored in a RAM as an array without using pointers.

Problem-92 Given a binary tree, find the maximum path sum. The path may start and end at any node in the tree.

Solution:

```

int maxPathSum(struct BinaryTreeNode *root){
    int maxValue = INT_MIN;
    return maxPathSumRec(root);
}
int max(int a, int b){
    if (a>b) return a;
    else return b;
}
int maxPathSumRec(struct BinaryTreeNode *root){
    if (root == NULL) return 0;
    int leftSum = maxPathSumRec(root->left);
    int rightSum = maxPathSumRec(root->right);
    if (leftSum < 0 && rightSum < 0){
        maxValue = max(maxValue, root->data);
        return root->data;
    }
    if (leftSum>0 && rightSum>0)
        maxValue = max(maxValue, root->data + leftSum + rightSum);
    maxValueUp = max(leftSum, rightSum) + root->data;
    maxValue = max(maxValue, maxValueUp);
    return maxValueUp;
}

```

Problem-93 Let T be a proper binary tree with root r. Consider the following algorithm.

```

Algorithm TreeTraversal(r):
    if (!r) return 1;
    else {
        a = TreeTraversal(r->left);
        b = TreeTraversal(r->right);
        return a + b;
    }

```

What does the algorithm do?

- A. It always returns the value 1.
- B. It computes the number of nodes in the tree.

- C. It computes the depth of the nodes.
- D. It computes the height of the tree.
- E. It computes the number of leaves in the tree.

Solution: E.

6.14 Other Variations on Trees

In this section, let us enumerate the other possible representations of trees. In the earlier sections, we have looked at AVL trees, which is a binary search tree (BST) with balancing property. Now, let us look at a few more balanced binary search trees: Red-black Trees and Splay Trees.

6.14.1 Red-Black Trees

In Red-black trees each node is associated with an extra attribute: the color, which is either red or black. To get logarithmic complexity we impose the following restrictions.

Definition: A Red-black tree is a binary search tree that satisfies the following properties:

- Root Property: the root is black
- External Property: every leaf is black
- Internal Property: the children of a red node are black
- Depth Property: all the leaves have the same black

Similar to AVL trees, if the Red-black tree becomes imbalanced, then we perform rotations to reinforce the balancing property. With Red-black trees, we can perform the following operations in $O(\log n)$ in worst case, where n is the number of nodes in the trees.

- Insertion, Deletion
- Finding predecessor, successor
- Finding minimum, maximum

6.14.2 Splay Trees

Splay-trees are BSTs with a self-adjusting property. Another interesting property of splay-trees is: starting with an empty tree, any sequence of K operations with maximum of n nodes takes $O(K \log n)$ time complexity in worst case. Splay trees are easier to program and also ensure faster access to recently accessed items. Similar to AVL and Red-Black trees, at any point that the splay tree becomes imbalanced, we can perform rotations to reinforce the balancing property.

Splay-trees cannot guarantee the $O(\log n)$ complexity in worst case. But it gives amortized $O(\log n)$ complexity. Even though individual operations can be expensive, any sequence of operations gets the complexity of logarithmic behavior. One operation may take more time (a

single operation may take $O(n)$ time) but the subsequent operations may not take worst case complexity and on the average *per operation* complexity is $O\{\log n\}$.

6.14.3 B-Trees

B-Tree is like other self-balancing trees such as AVL and Red-black tree such that it maintains its balance of nodes while operations are performed against it. B-Tree has the following properties:

- Minimum degree “ t ” where, except root node, all other nodes must have no less than $t - 1$ keys
- Each node with n keys has $n + 1$ children
- Keys in each node are lined up where $k_1 < k_2 < \dots < k_n$
- Each node cannot have more than $2t-1$ keys, thus $2t$ children
- Root node at least must contain one key. There is no root node if the tree is empty.
- Tree grows in depth only when root node is split.

Unlike a binary-tree, each node of a b-tree may have a variable number of keys and children. The keys are stored in non-decreasing order. Each key has an associated child that is the root of a subtree containing all nodes with keys less than or equal to the key but greater than the preceding key. A node also has an additional rightmost child that is the root for a subtree containing all keys greater than any keys in the node.

A b-tree has a minimum number of allowable children for each node known as the *minimization factor*. If t is this *minimization factor*, every node must have at least $t - 1$ keys. Under certain circumstances, the root node is allowed to violate this property by having fewer than $t - 1$ keys. Every node may have at most $2t - 1$ keys or, equivalently, $2t$ children.

Since each node tends to have a large branching factor (a large number of children), it is typically necessary to traverse relatively few nodes before locating the desired key. If access to each node requires a disk access, then a B-tree will minimize the number of disk accesses required. The minimization factor is usually chosen so that the total size of each node corresponds to a multiple of the block size of the underlying storage device. This choice simplifies and optimizes disk access. Consequently, a B-tree is an ideal data structure for situations where all data cannot reside in primary storage and accesses to secondary storage are comparatively expensive (or time consuming).

To *search* the tree, it is similar to binary tree except that the key is compared multiple times in a given node because the node contains more than 1 key. If the key is found in the node, the search terminates. Otherwise, it moves down where a child pointed to by c_i where $\text{key } k < k_i$.

Key *insertions* of a B-tree happens from the bottom fashion. This means that it walk down the tree from root to the target child node first. If the child is not full, the key is simply inserted. If it is full, the child node is split in the middle, the median key moves up to the parent, then the new key

is inserted. When inserting and walking down the tree, if the root node is found to be full, it's split first and we have a new root node. Then the normal insertion operation is performed.

Key *deletion* is more complicated as it needs to maintain the number of keys in each node to meet the constraint. If a key is found in leaf node and deleting it still keeps the number of keys in the nodes not too low, it's simply done right away. If it's done to the inner node, the predecessor of the key in the corresponding child node is moved to replace the key in the inner node. If moving the predecessor will cause the child node to violate the node count constraint, the sibling child nodes are combined and the key in the inner node is deleted.

6.14.4 Augmented Trees

In earlier sections, we have seen various problems like finding the K^{th} - smallest - element in the tree and other similar ones. Of all the problems the worst complexity is $O(n)$, where n is the number of nodes in the tree. To perform such operations in $O(\log n)$, augmented trees are useful. In these trees, extra information is added to each node and that extra data depends on the problem we are trying to solve.

For example, to find the K^{th} element in a binary search tree, let us see how augmented trees solve the problem. Let us assume that we are using Red-Black trees as balanced BST (or any balanced BST) and augmenting the size information in the nodes data. For a given node X in Red-Black tree with a field $size(X)$ equal to the number of nodes in the subtree and can be calculated as:

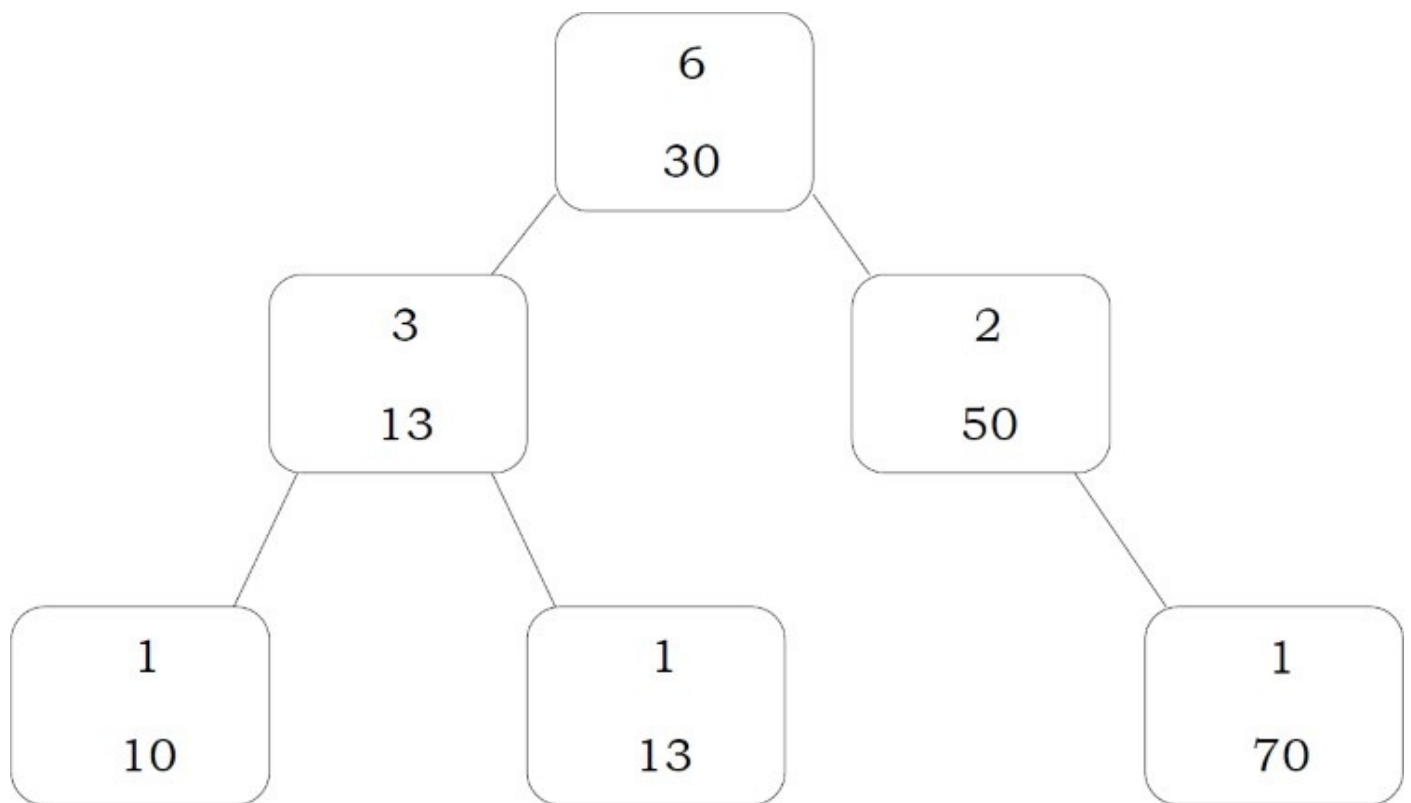
$$size(X) = size(X \rightarrow left) + size(X \rightarrow right) + 1$$

K^{th} - smallest - operation can be defined as:

```
struct BinarySearchTreeNode *KthSmallest (struct BinarySearchTreeNode *X, int K) {  
    int r = size(X→left) + 1;  
    if(K == r)  
        return X;  
    if(K < r)  
        return KthSmallest (X→left, K);  
    if(K > r)  
        return KthSmallest (X→right, K-r);  
}
```

Time Complexity: $O(\log n)$. Space Complexity: $O(\log n)$.

Example: With the extra size information, the augmented tree will look like:



6.14.5 Interval Trees [Segment Trees]

We often face questions that involve queries made in an array based on range. For example, for a given array of integers, what is the maximum number in the range α to β , where α and β are of course within array limits. To iterate over those entries with intervals containing a particular value, we can use a simple array. But if we need more efficient access, we need a more sophisticated data structure.

An array-based storage scheme and a brute-force search through the entire array is acceptable only if a single search is to be performed, or if the number of elements is small. For example, if you know all the array values of interest in advance, you need to make only one pass through the array. However, if you can interactively specify different search operations at different times, the brute-force search becomes impractical because every element in the array must be examined during each search operation.

If you sort the array in ascending order of the array values, you can terminate the sequential search when you reach the object whose low value is greater than the element we are searching. Unfortunately, this technique becomes increasingly ineffective as the low value increases, because fewer search operations are eliminated. That means, what if we have to answer a large number of queries like this? – is brute force still a good option?

Another example is when we need to return a sum in a given range. We can brute force this too, but the problem for a large number of queries still remains. So, what can we do? With a bit of thinking we can come up with an approach like maintaining a separate array of n elements, where

n is the size of the original array, where each index stores the sum of all elements from 0 to that index. So essentially we have with a bit of preprocessing brought down the query time from a worst case $O(n)$ to $O(1)$. Now this is great as far as static arrays are concerned, but, what if we are required to perform updates on the array too?

The first approach gives us an $O(n)$ query time, but an $O(1)$ update time. The second approach, on the other hand, gives us $O(1)$ query time, but an $O(n)$ update time. So, which one do we choose?

Interval trees are also binary search trees and they store interval information in the node structure. That means, we maintain a set of n intervals $[i_1, i_2]$ such that one of the intervals containing a query point Q (if any) can be found efficiently. Interval trees are used for performing range queries efficiently.

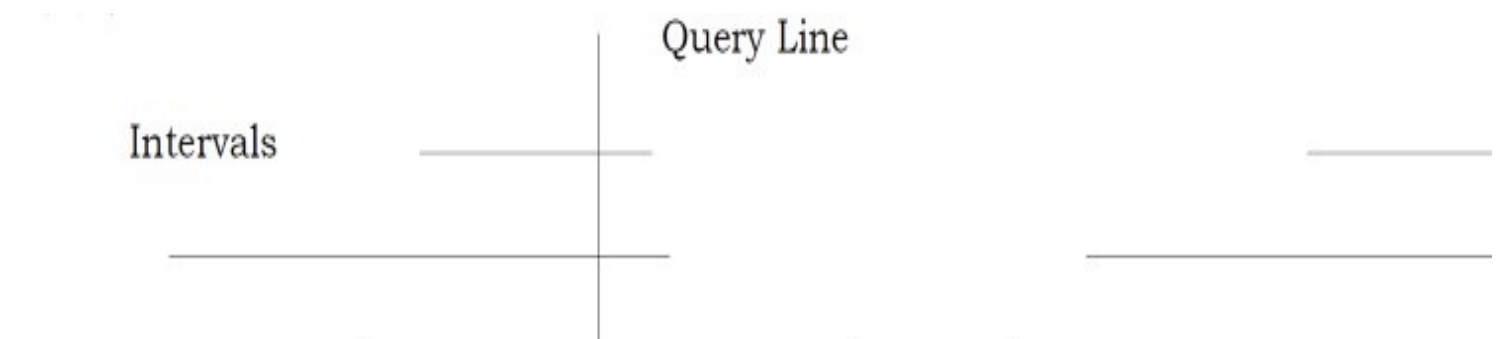
A segment tree is a heap-like data structure that can be used for making update/query operations upon array intervals in logarithmical time. We define the segment tree for the interval $[i, j]$ in the following recursive manner:

- The root (first node in the array) node will hold the information for the interval $[i, j]$
- If $i < j$ the left and right children will hold the information for the intervals $[i, \frac{i+j}{2}]$ and $[\frac{i+j}{2} + 1, j]$

Segment trees (also called *seg*trees and *interval trees*) is a cool data structure, primarily used for range queries. It is a height balanced binary tree with a static structure. The nodes of a segment tree correspond to various intervals, and can be augmented with appropriate information pertaining to those intervals. It is somewhat less powerful than a balanced binary tree because of its static structure, but due to the recursive nature of operations on the segtree, it is incredibly easy to think about and code.

We can use segment trees to solve range minimum/maximum query problems. The time complexity is $T(n \log n)$ where $O(n)$ is the time required to build the tree and each query takes $O(\log n)$ time.

Example: Given a set of intervals: $S = \{[2-5], [6-7], [6-10], [8-9], [12-15], [15-23], [25-30]\}$. A query with $Q = 9$ returns $[6,10]$ or $[8,9]$ (assume these are the intervals which contain 9 among all the intervals). A query with $Q = 23$ returns $[15, 23]$.



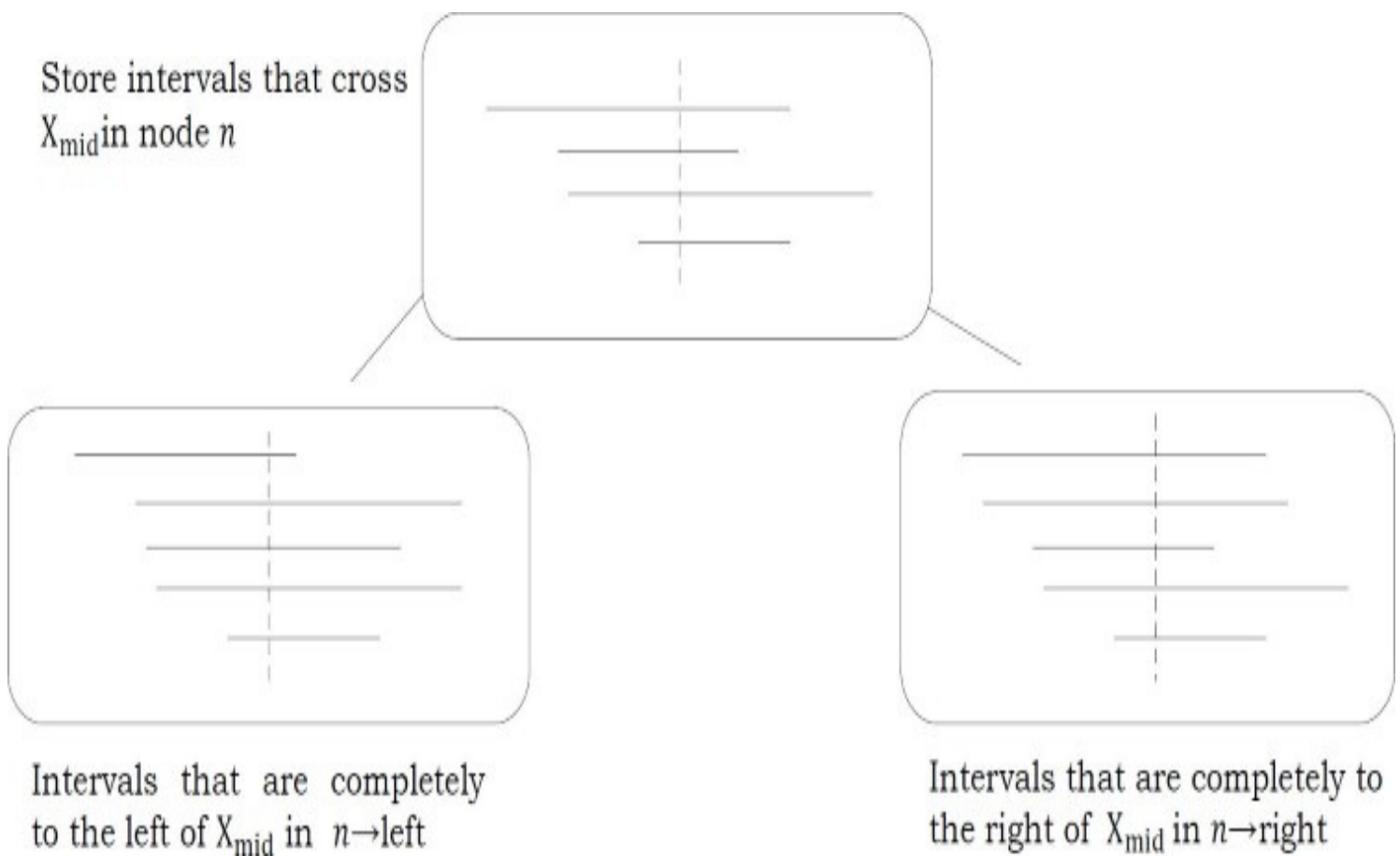
Construction of Interval Trees: Let us assume that we are given a set S of n intervals (called *segments*). These n intervals will have $2n$ endpoints. Now, let us see how to construct the interval tree.

Algorithm:

Recursively build tree on interval set S as follows:

- Sort the $2n$ endpoints
- Let X_{mid} be the median point

Time Complexity for building interval trees: $O(n \log n)$. Since we are choosing the median, Interval Trees will be approximately balanced. This ensures that, we split the set of end points up in half each time. The depth of the tree is $O(\log n)$. To simplify the search process, generally X_{mid} is stored with each node.



6.14.6 Scapegoat Trees

Scapegoat tree is a self-balancing binary search tree, discovered by Arne Andersson. It provides worst-case $O(\log n)$ search time, and $O(\log n)$ amortized (average) insertion and deletion time.

AVL trees rebalance whenever the height of two sibling subtrees differ by more than one;

scapegoat trees rebalance whenever the size of a child exceeds a certain ratio of its parents, a ratio known as α . After inserting the element, we traverse back up the tree. If we find an imbalance where a child's size exceeds the parent's size times α , we must rebuild the subtree at the parent, the *scapegoat*.

There might be more than one possible scapegoat, but we only have to pick one. The most optimal scapegoat is actually determined by height balance. When removing it, we see if the total size of the tree is less than α of the largest size since the last rebuilding of the tree. If so, we rebuild the entire tree. The α for a scapegoat tree can be any number between 0.5 and 1.0. The value 0.5 will force perfect balance, while 1.0 will cause rebalancing to never occur, effectively turning it into a BST.

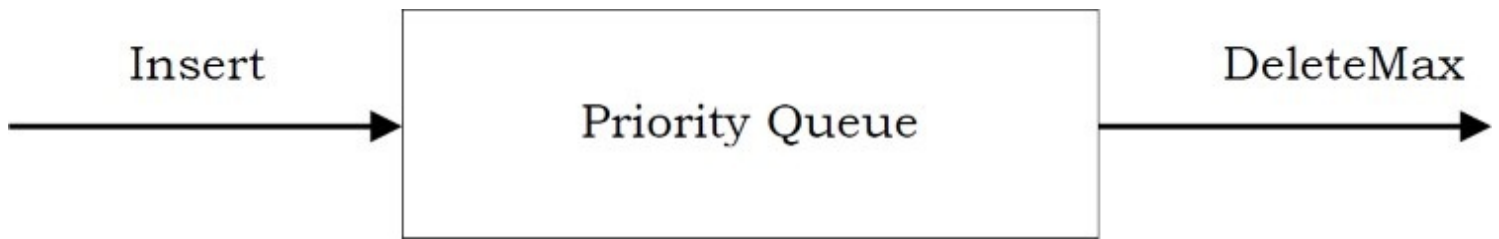


PRIORITY QUEUES AND HEAPS

7.1 What is a Priority Queue?

In some situations we may need to find the minimum/maximum element among a collection of elements. We can do this with the help of Priority Queue ADT. A priority queue ADT is a data structure that supports the operations *Insert* and *DeleteMin* (which returns and removes the minimum element) or *DeleteMax* (which returns and removes the maximum element).

These operations are equivalent to *EnQueue* and *DeQueue* operations of a queue. The difference is that, in priority queues, the order in which the elements enter the queue may not be the same in which they were processed. An example application of a priority queue is job scheduling, which is prioritized instead of serving in first come first serve.



A priority queue is called an *ascending – priority* queue, if the item with the smallest key has the highest priority (that means, delete the smallest element always). Similarly, a priority queue is said to be a *descending –priority* queue if the item with the largest key has the highest priority (delete the maximum element always). Since these two types are symmetric we will be concentrating on one of them: ascending-priority queue.

7.2 Priority Queue ADT

The following operations make priority queues an ADT.

Main Priority Queues Operations

A priority queue is a container of elements, each having an associated key.

- Insert (key, data): Inserts data with *key* to the priority queue. Elements are ordered based on key.
- DeleteMin/DeleteMax: Remove and return the element with the smallest/largest key.
- GetMinimum/GetMaximum: Return the element with the smallest/largest key without deleting it.

Auxiliary Priority Queues Operations

- k^{th} - Smallest/ k^{th} – Largest: Returns the k^{th} -Smallest/ k^{th} –Largest key in priority queue.
- Size: Returns number of elements in priority queue.
- Heap Sort: Sorts the elements in the priority queue based on priority (key).

7.3 Priority Queue Applications

Priority queues have many applications - a few of them are listed below:

- Data compression: Huffman Coding algorithm
- Shortest path algorithms: Dijkstra’s algorithm
- Minimum spanning tree algorithms: Prim’s algorithm
- Event-driven simulation: customers in a line

- Selection problem: Finding k^{th} - smallest element

7.4 Priority Queue Implementations

Before discussing the actual implementation, let us enumerate the possible options.

Unordered Array Implementation

Elements are inserted into the array without bothering about the order. Deletions (DeleteMax) are performed by searching the key and then deleting.

Insertions complexity: $O(1)$. DeleteMin complexity: $O(n)$.

Unordered List Implementation

It is very similar to array implementation, but instead of using arrays, linked lists are used.

Insertions complexity: $O(1)$. DeleteMin complexity: $O(n)$.

Ordered Array Implementation

Elements are inserted into the array in sorted order based on key field. Deletions are performed at only one end.

Insertions complexity: $O(n)$. DeleteMin complexity: $O(1)$.

Ordered List Implementation

Elements are inserted into the list in sorted order based on key field. Deletions are performed at only one end, hence preserving the status of the priority queue. All other functionalities associated with a linked list ADT are performed without modification.

Insertions complexity: $O(n)$. DeleteMin complexity: $O(1)$.

Binary Search Trees Implementation

Both insertions and deletions take $O(\log n)$ on average if insertions are random (refer to [Trees](#) chapter).

Balanced Binary Search Trees Implementation

Both insertions and deletion take $O(\log n)$ in the worst case (refer to [Trees](#) chapter).

Binary Heap Implementation

In subsequent sections we will discuss this in full detail. For now, assume that binary heap implementation gives $O(\log n)$ complexity for search, insertions and deletions and $O(1)$ for finding the maximum or minimum element.

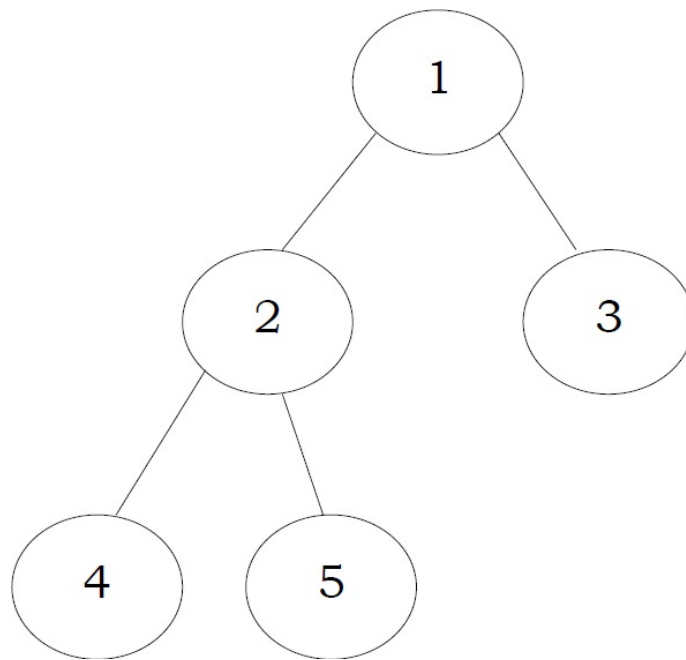
Comparing Implementations

Implementation	Insertion	Deletion (DeleteMax)	Find Min
Unordered array	1	n	n
Unordered list	1	n	n
Ordered array	n	1	1
Ordered list	n	1	1
Binary Search Trees	$\log n$ (average)	$\log n$ (average)	$\log n$ (average)
Balanced Binary Search Trees	$\log n$	$\log n$	$\log n$
Binary Heaps	$\log n$	$\log n$	1

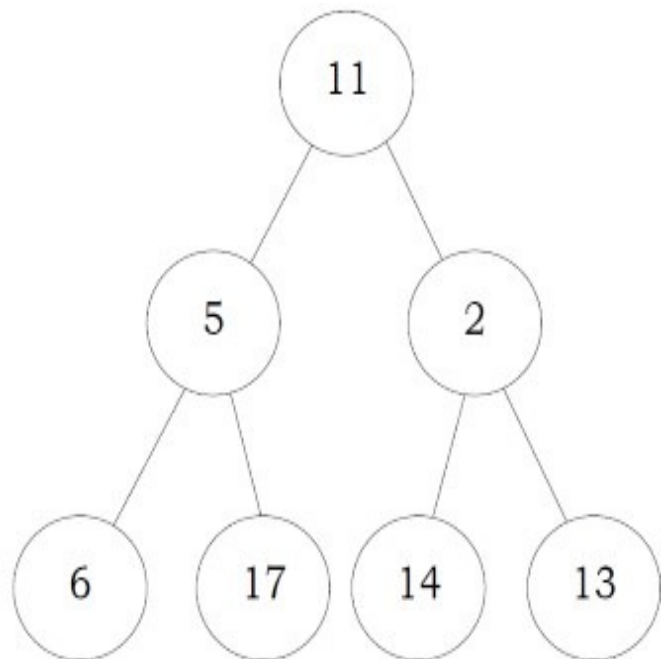
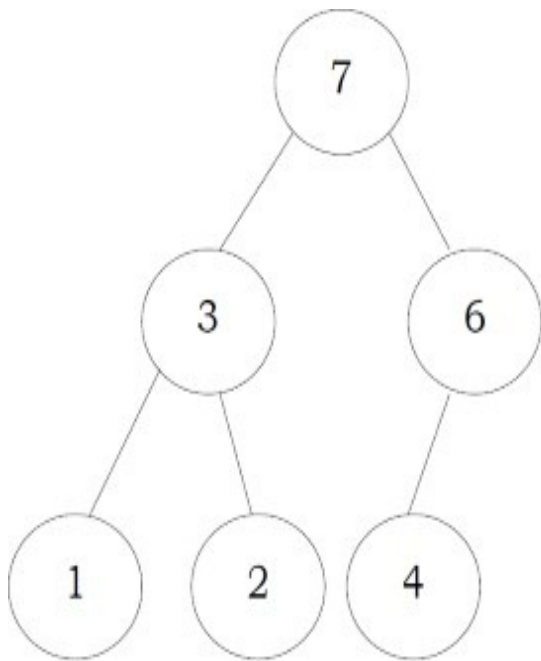
7.5 Heaps and Binary Heaps

What is a Heap?

A heap is a tree with some special properties. The basic requirement of a heap is that the value of a node must be $\geq$ (or $\leq$) than the values of its children. This is called *heap property*. A heap also has the additional property that all leaves should be at h or $h - 1$ levels (where h is the height of the tree) for some $h > 0$ (*complete binary trees*). That means heap should form a *complete binary tree* (as shown below).



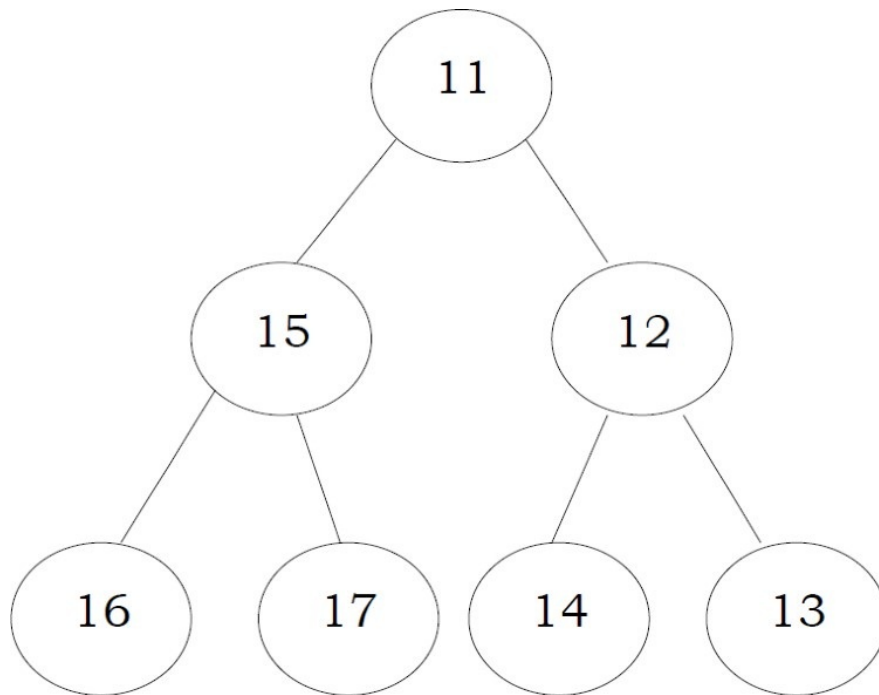
In the examples below, the left tree is a heap (each element is greater than its children) and the right tree is not a heap (since 11 is greater than 2).



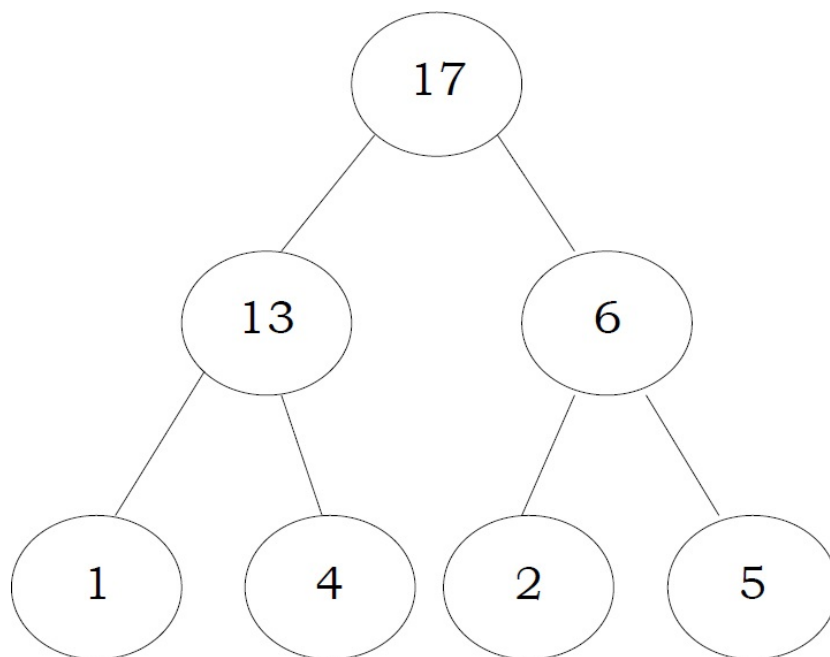
Types of Heaps?

Based on the property of a heap we can classify heaps into two types:

- **Min heap:** The value of a node must be less than or equal to the values of its children



- **Max heap:** The value of a node must be greater than or equal to the values of its children



7.6 Binary Heaps

In binary heap each node may have up to two children. In practice, binary heaps are enough and we concentrate on binary min heaps and binary max heaps for the remaining discussion.

Representing Heaps: Before looking at heap operations, let us see how heaps can be represented. One possibility is using arrays. Since heaps are forming complete binary trees, there will not be any wastage of locations. For the discussion below let us assume that elements are

stored in arrays, which starts at index 0. The previous max heap can be represented as:

17	13	6	1	4	2	5
0	1	2	3	4	5	6

Note: For the remaining discussion let us assume that we are doing manipulations in max heap.

Declaration of Heap

```
struct Heap {  
    int *array;  
    int count;           // Number of elements in Heap  
    int capacity;        // Size of the heap  
    int heap_type;       // Min Heap or Max Heap  
};
```

Creating Heap

```

struct Heap * CreateHeap(int capacity, int heap_type) {
    struct Heap * h = (struct Heap *)malloc(sizeof(struct Heap));
    if(h == NULL) {
        printf("Memory Error");
        return;
    }
    h->heap_type = heap_type;
    h->count = 0;
    h->capacity = capacity;
    h->array = (int *) malloc(sizeof(int) * h->capacity);
    if(h->array == NULL) {
        printf("Memory Error");
        return;
    }
    return h;
}

```

Time Complexity: $O(1)$.

Parent of a Node

For a node at i^{th} location, its parent is at $\frac{i-1}{2}$ location. In the previous example, the element 6 is at second location and its parent is at 0^{th} location.

```

int Parent (struct Heap * h, int i) {
    if(i <= 0 || i >= h->count)
        return -1;
    return i-1/2;
}

```

Time Complexity: $O(1)$.

Children of a Node

Similar to the above discussion, for a node at i^{th} location, its children are at $2 * i + 1$ and $2 * i +$

2 locations. For example, in the above tree the element 6 is at second location and its children 2 and 5 are at 5 ($2 * i + 1 = 2 * 2 + 1$) and 6 ($2 * i + 2 = 2 * 2$) locations.

```
int LeftChild(struct Heap *h, int i) {  
    int left = 2 * i + 1;  
    if(left >= h->count)  
        return -1;  
    return left;  
}
```

Time Complexity: $O(1)$.

```
int RightChild(struct Heap *h, int i) {  
    int right = 2 * i + 2;  
    if(right >= h->count)  
        return -1;  
    return right;  
}
```

Time Complexity: $O(1)$.

Getting the Maximum Element

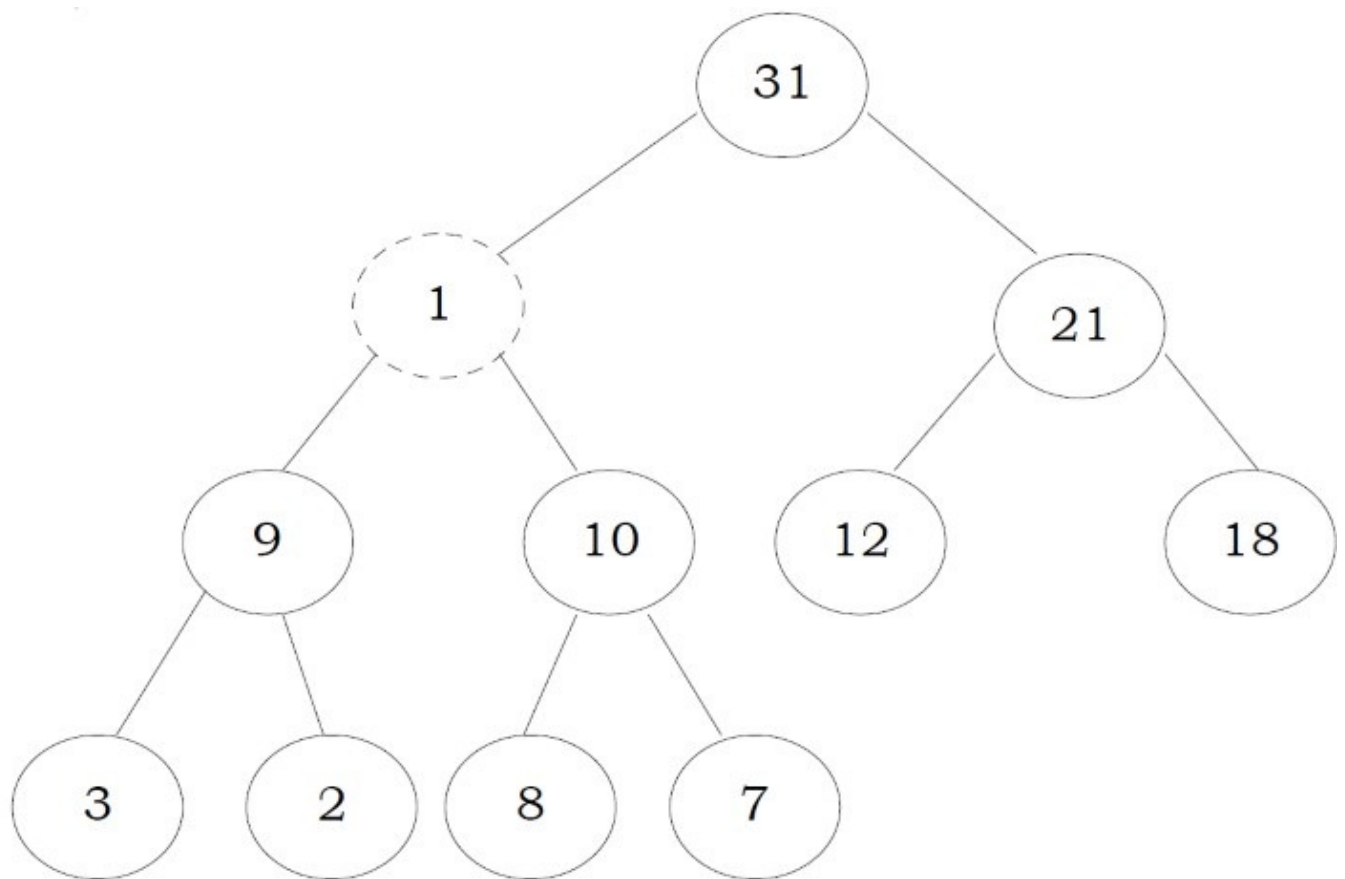
Since the maximum element in max heap is always at root, it will be stored at $h \rightarrow \text{array}[0]$.

```
int GetMaximum(Heap * h) {  
    if(h->count == 0)  
        return -1;  
    return h->array[0];  
}
```

Time Complexity: $O(1)$.

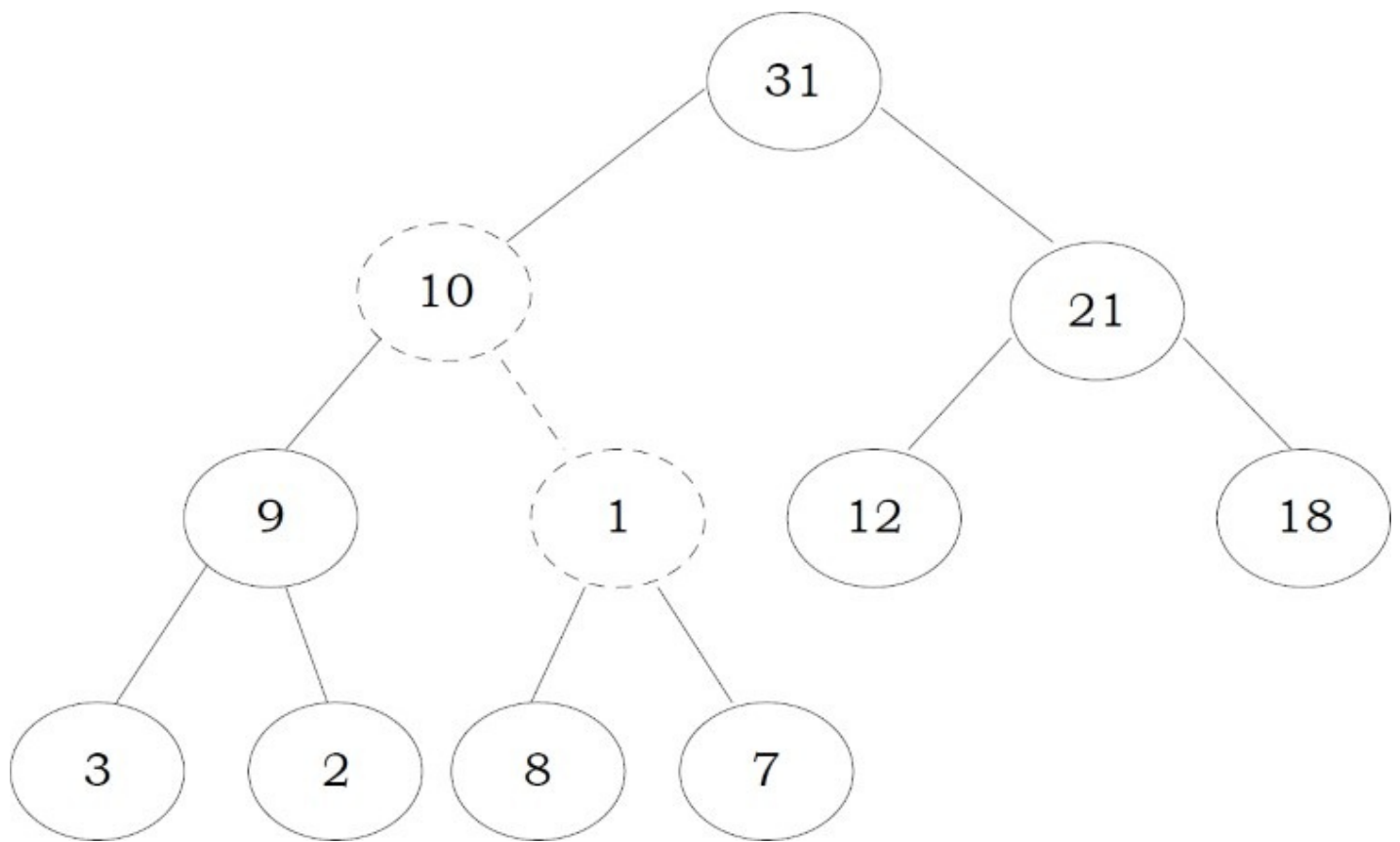
Heapifying an Element

After inserting an element into heap, it may not satisfy the heap property. In that case we need to adjust the locations of the heap to make it heap again. This process is called *heapifying*. In max-heap, to heapify an element, we have to find the maximum of its children and swap it with the current element and continue this process until the heap property is satisfied at every node.

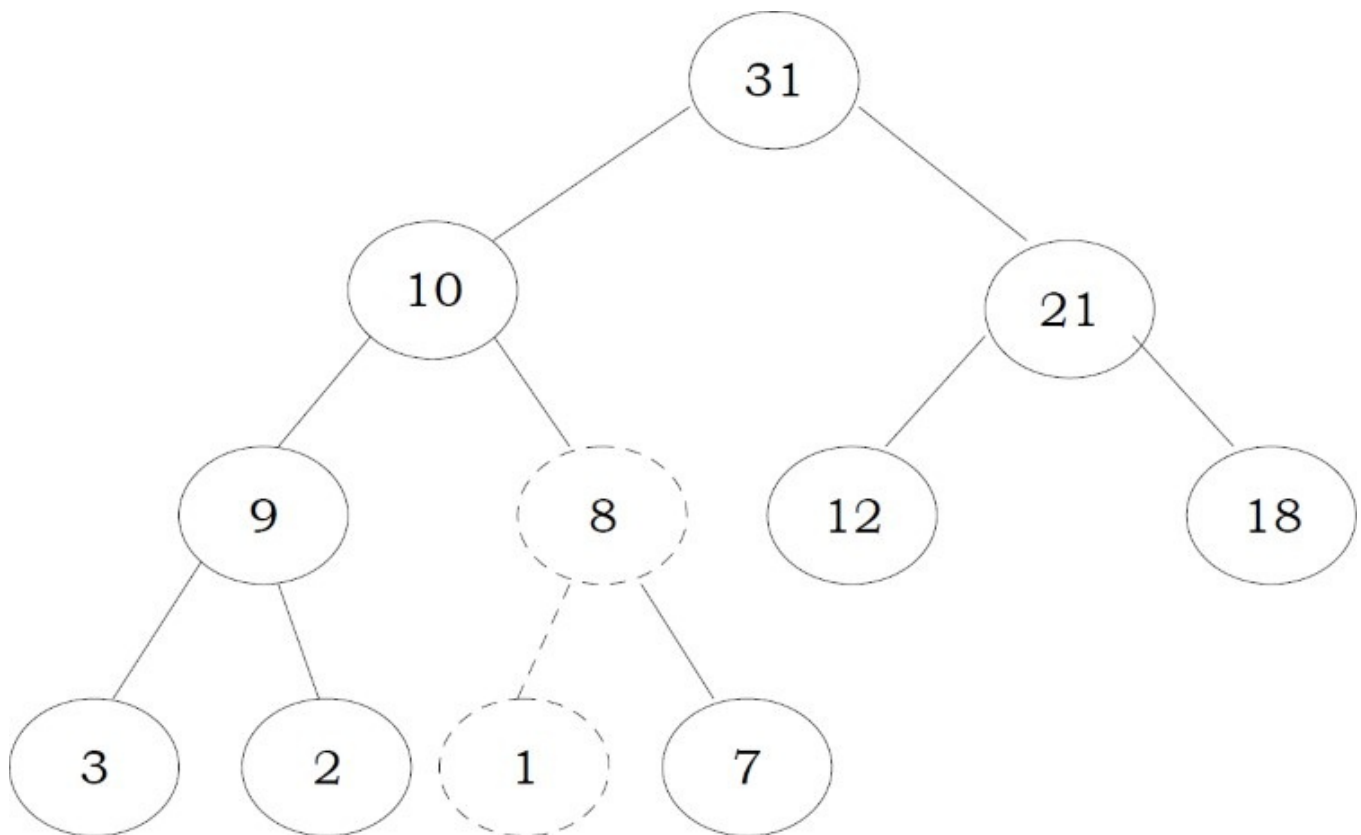


Observation: One important property of heap is that, if an element is not satisfying the heap property, then all the elements from that element to the root will have the same problem. In the example below, element 1 is not satisfying the heap property and its parent 31 is also having the issue. Similarly, if we heapify an element, then all the elements from that element to the root will also satisfy the heap property automatically. Let us go through an example. In the above heap, the element 1 is not satisfying the heap property. Let us try heapifying this element.

To heapify 1, find the maximum of its children and swap with that.



We need to continue this process until the element satisfies the heap properties. Now, swap 1 with 8.



Now the tree is satisfying the heap property. In the above heapifying process, since we are

moving from top to bottom, this process is sometimes called *percolate down*. Similarly, if we start heapifying from any other node to root, we can that process *percolate up* as move from bottom to top.

```
//Heapifying the element at location i.
void PercolateDown(struct Heap *h, int i) {
    int l, r, max, temp;
    l = LeftChild(h, i);
    r = RightChild(h, i);
    if(l != -1 && h->array[l] > h->array[i])
        max = l;
    else
        max = i;
    if(r != -1 && h->array[r] > h->array[max])
        max = r;
    if(max != i) {
        //Swap h->array[i] and h->array[max];
        temp = h->array[i];
        h->array[i] = h->array[max];
        h->array[max] = temp;
    }
    PercolateDown(h, max);
}
```

Time Complexity: $O(\log n)$. Heap is a complete binary tree and in the worst case we start at the root and come down to the leaf. This is equal to the height of the complete binary tree. Space Complexity: $O(1)$.

Deleting an Element

To delete an element from heap, we just need to delete the element from the root. This is the only operation (maximum element) supported by standard heap. After deleting the root element, copy the last element of the heap (tree) and delete that last element.

After replacing the last element, the tree may not satisfy the heap property. To make it heap again, call the *PercolateDown* function.

- Copy the first element into some variable

- Copy the last element into first element location
- *PercolateDown* the first element

```
int DeleteMax(struct Heap *h) {  
    int data;  
    if(h->count == 0)  
        return -1;  
    data = h->array[0];  
    h->array[0] = h->array[h->count-1];  
    h->count--; //reducing the heap size  
    PercolateDown(h, 0);  
    return data;  
}
```

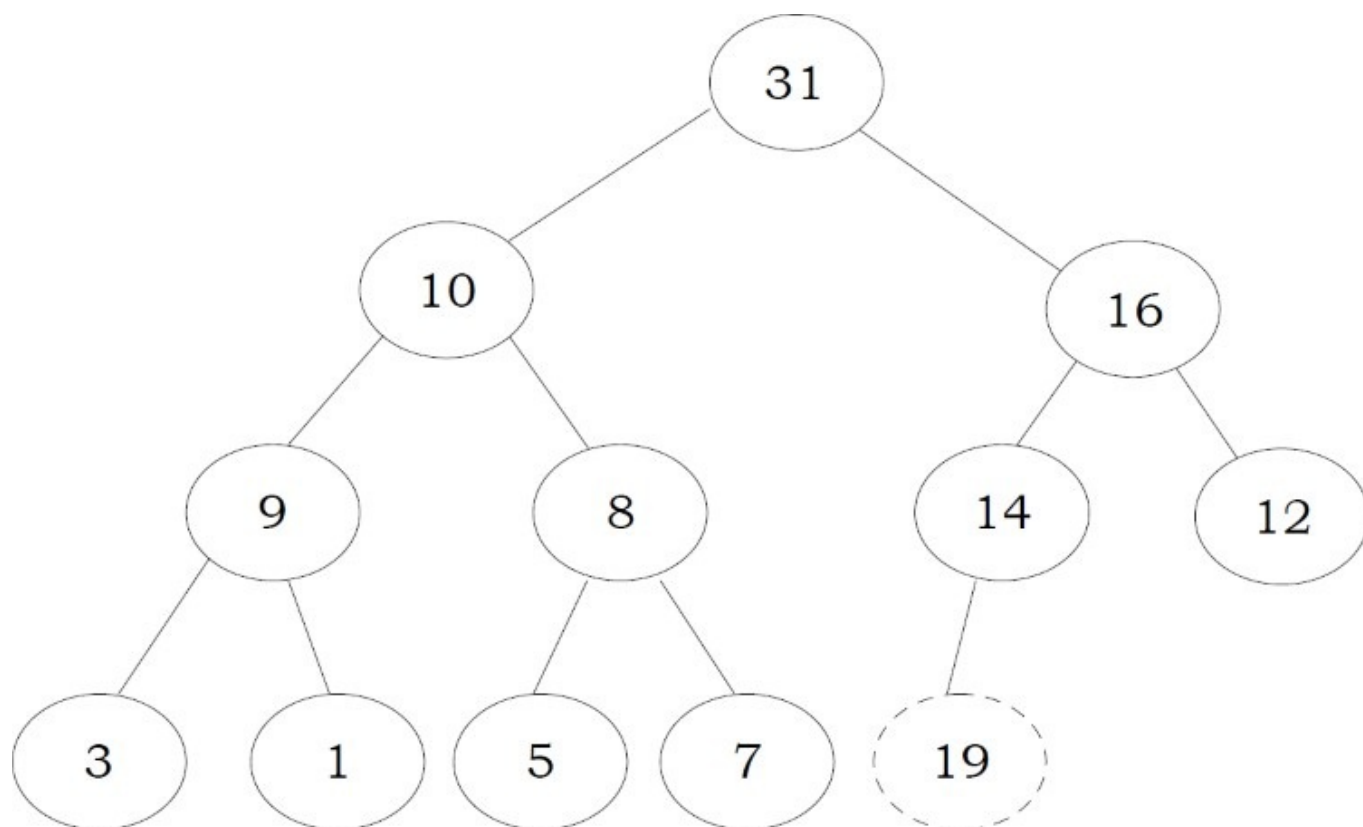
Note: Deleting an element uses *PercolateDown*, and inserting an element uses *PercolateUp*.
Time Complexity: same as *Heapify* function and it is $O(\log n)$.

Inserting an Element

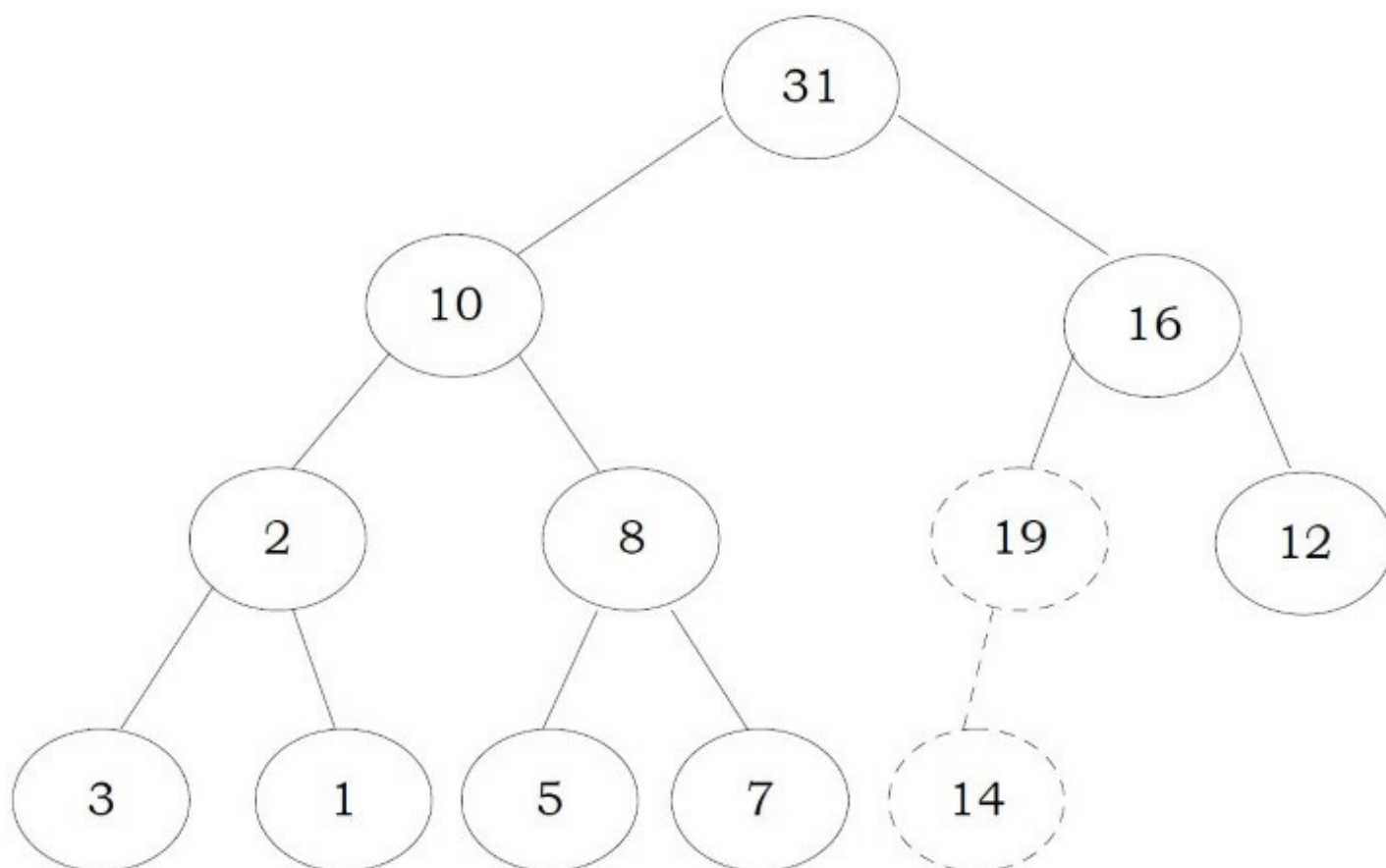
Insertion of an element is similar to the heapify and deletion process.

- Increase the heap size
- Keep the new element at the end of the heap (tree)
- Heapify the element from bottom to top (root)

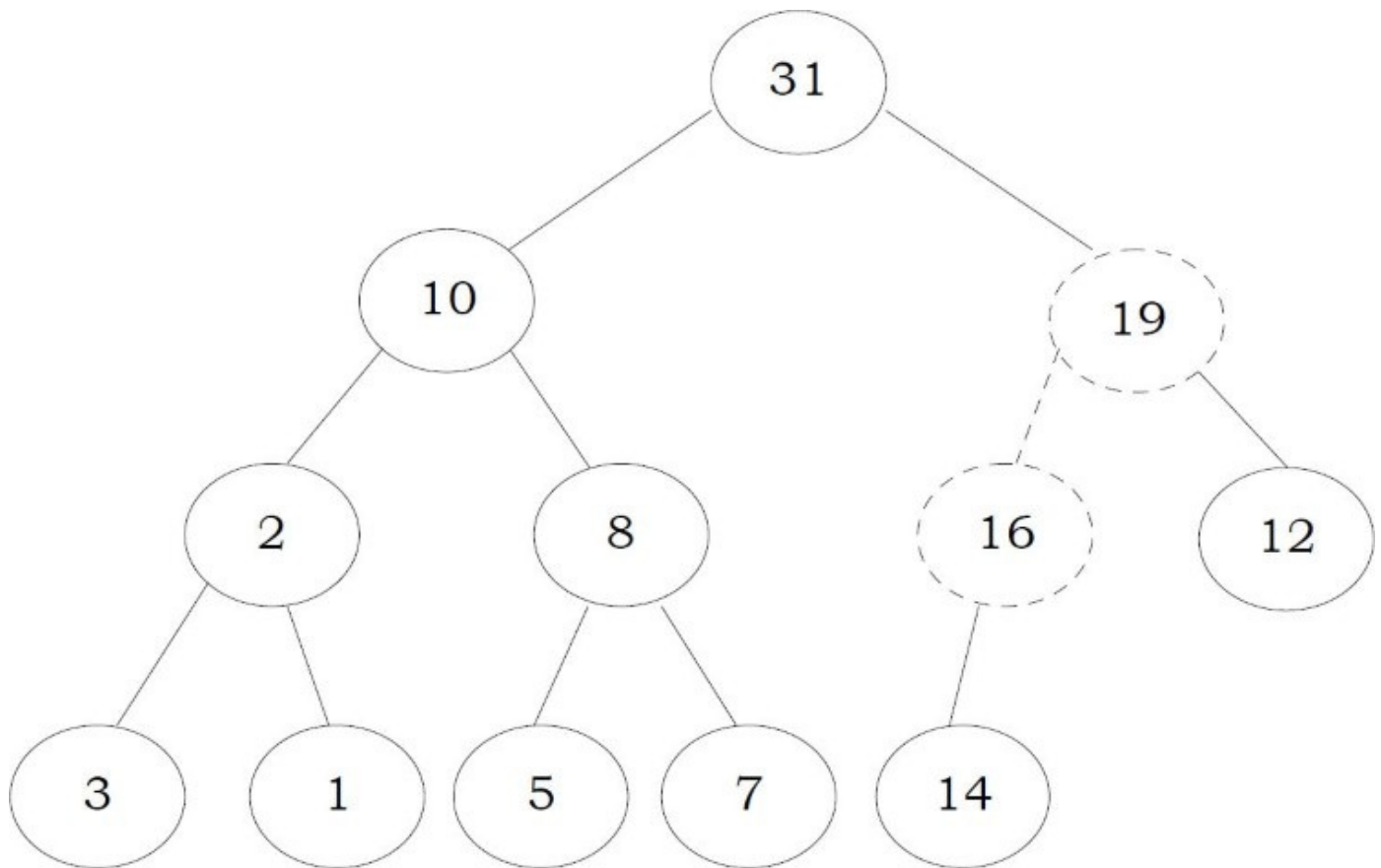
Before going through code, let us look at an example. We have inserted the element 19 at the end of the heap and this is not satisfying the heap property.



In order to heapify this element (19), we need to compare it with its parent and adjust them. Swapping 19 and 14 gives:



Again, swap 19 and 16:



Now the tree is satisfying the heap property. Since we are following the bottom-up approach we sometimes call this process *percolate up*.

```

int Insert(struct Heap *h, int data) {
    int i;
    if(h->count == h->capacity)
        ResizeHeap(h);
    h->count++;          //increasing the heap size to hold this new item
    i = h->count-1;
    while(i >= 0 && data > h->array[(i-1)/2]) {
        h->array[i] = h->array[(i-1)/2];
        i = i-1/2;
    }
    h->array[i] = data;
}

void ResizeHeap(struct Heap * h) {
    int *array_old = h->array;
    h->array = (int *) malloc(sizeof(int) * h->capacity * 2);
    if(h->array == NULL) {
        printf("Memory Error");
        return;
    }
    for (int i = 0; i < h->capacity; i++)
        h->array[i] = array_old[i];
    h->capacity *= 2;
    free(array_old);
}

```

Time Complexity: $O(\log n)$. The explanation is the same as that of the *Heapify* function.

Destroying Heap

```

void DestroyHeap (struct Heap *h) {
    if(h == NULL)
        return;
    free(h->array);
    free(h);
    h = NULL;
}

```

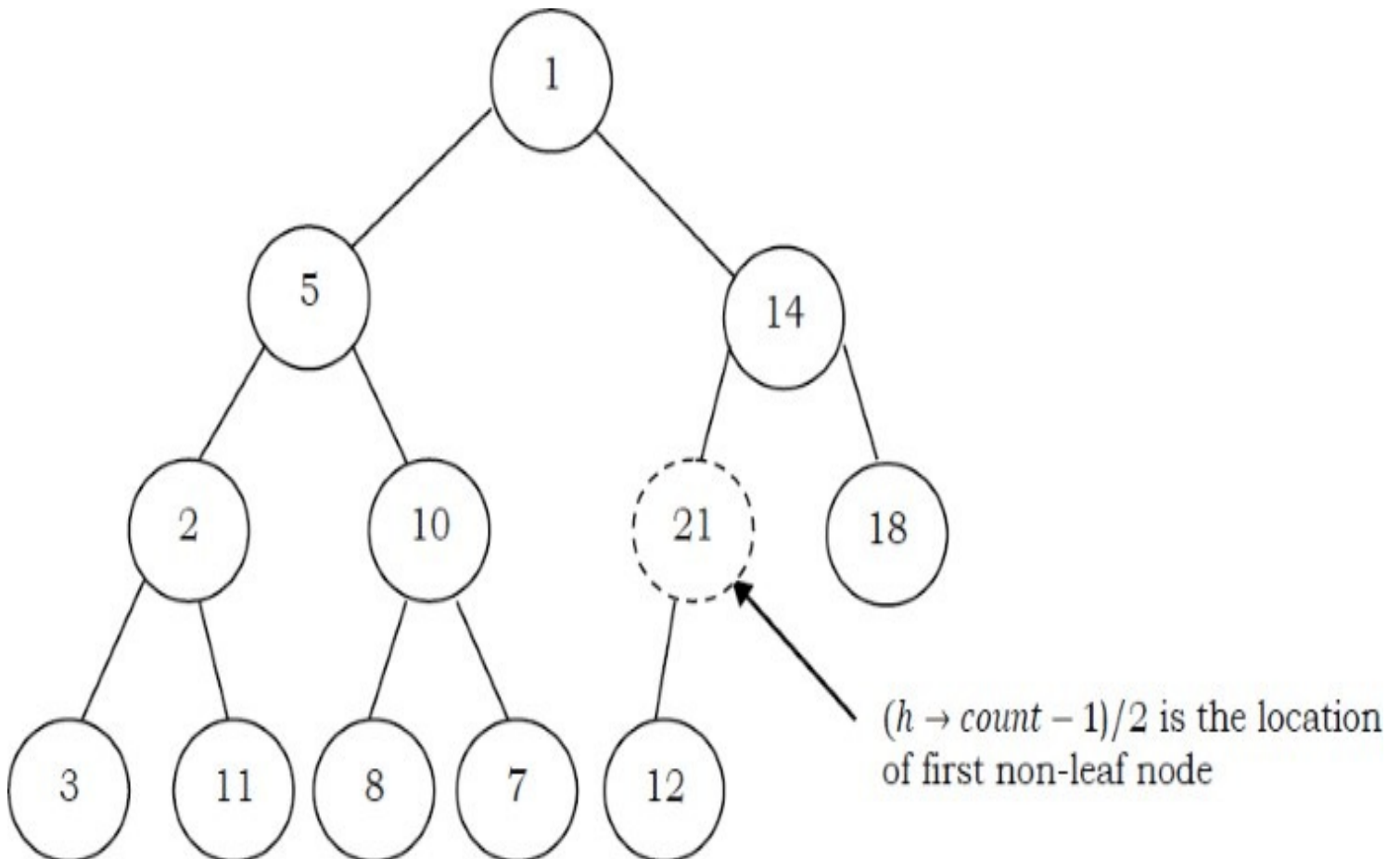
Heapifying the Array

One simple approach for building the heap is, take n input items and place them into an empty heap. This can be done with n successive inserts and takes $O(n \log n)$ in the worst case. This is due to the fact that each insert operation takes $O(\log n)$.

To finish our discussion of binary heaps, we will look at a method to build an entire heap from a list of keys. The first method you might think of may be like the following. Given a list of keys, you could easily build a heap by inserting each key one at a time. Since you are starting with a list of one item, the list is sorted and you could use binary search to find the right position to insert the next key at a cost of approximately $O(\log n)$ operations.

However, remember that inserting an item in the middle of the list may require $O(n)$ operations to shift the rest of the list over to make room for the new key. Therefore, to insert n keys into the heap would require a total of $O(n \log n)$ operations. However, if we start with an entire list then we can build the whole heap in $O(n)$ operations.

Observation: Leaf nodes always satisfy the heap property and do not need to care for them. The leaf elements are always at the end and to heapify the given array it should be enough if we heapify the non-leaf nodes. Now let us concentrate on finding the first non-leaf node. The last element of the heap is at location $h \rightarrow \text{count} - 1$, and to find the first non-leaf node it is enough to find the parent of the last element.



```

void BuildHeap(struct Heap *h, int A[], int n) {
    if(h == NULL)
        return;
    while (n > h->capacity)
        ResizeHeap(h);
    for (int i = 0; i < n; i++)
        h->array[i] = A[i];
    h->count = n;
    for (int i = (n-1)/2; i >= 0; i--)
        PercolateDown(h, i);
}

```

Time Complexity: The linear time bound of building heap can be shown by computing the sum of the heights of all the nodes. For a complete binary tree of height h containing $n = 2^{h+1} - 1$ nodes, the sum of the heights of the nodes is $n - h - 1 = n - \log n - 1$ (for proof refer to *Problems Section*). That means, building the heap operation can be done in linear time ($O(n)$) by applying a *PercolateDown* function to the nodes in reverse level order.

7.7 Heapsort

One main application of heap ADT is sorting (heap sort). The heap sort algorithm inserts all elements (from an unsorted array) into a heap, then removes them from the root of a heap until the heap is empty. Note that heap sort can be done in place with the array to be sorted. Instead of deleting an element, exchange the first element (maximum) with the last element and reduce the heap size (array size). Then, we heapify the first element. Continue this process until the number of remaining elements is one.

```

void Heapsort(int A[], in n) {
    struct Heap *h = CreateHeap(n);
    int old_size, i, temp;
    BuildHeap(h, A, n);
    old_size = h->count;
    for(i = n-1; i > 0; i--) {
        //h->array [0] is the largest element
        temp = h->array[0];
        h->array[0] = h->array[h->count-1];
        h->array[h->count-1] = temp;
        h->count--;
        PercolateDown(h, 0);
    }
    h->count = old_size;
}

```

Time complexity: As we remove the elements from the heap, the values become sorted (since maximum elements are always *root* only). Since the time complexity of both the insertion algorithm and deletion algorithm is $O(\log n)$ (where n is the number of items in the heap), the time complexity of the heap sort algorithm is $O(n \log n)$.

7.8 Priority Queues [Heaps]: Problems & Solutions

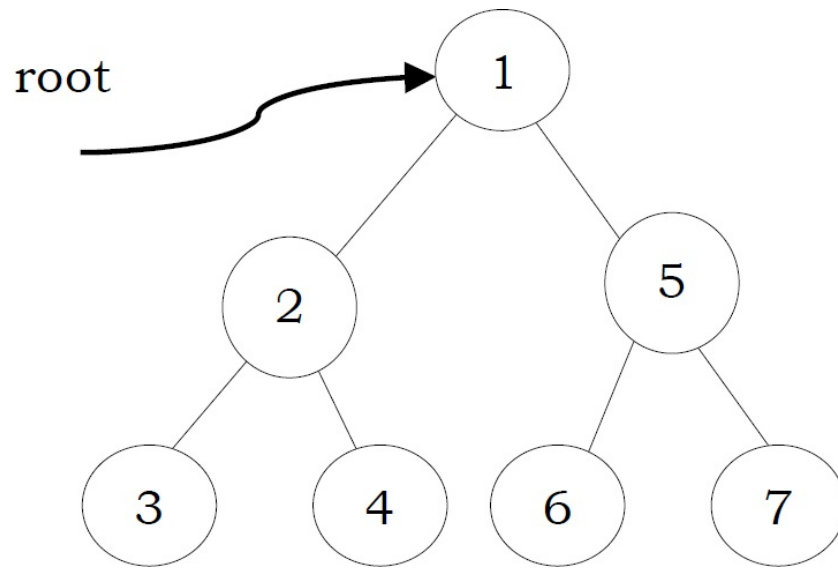
Problem-1 What are the minimum and maximum number of elements in a heap of height h ?

Solution: Since heap is a complete binary tree (all levels contain full nodes except possibly the lowest level), it has at most $2^{h+1} - 1$ elements (if it is complete). This is because, to get maximum nodes, we need to fill all the h levels completely and the maximum number of nodes is nothing but the sum of all nodes at all h levels.

To get minimum nodes, we should fill the $h - 1$ levels fully and the last level with only one element. As a result, the minimum number of nodes is nothing but the sum of all nodes from $h - 1$ levels plus 1 (for the last level) and we get $2^h - 1 + 1 = 2^h$ elements (if the lowest level has just 1 element and all the other levels are complete).

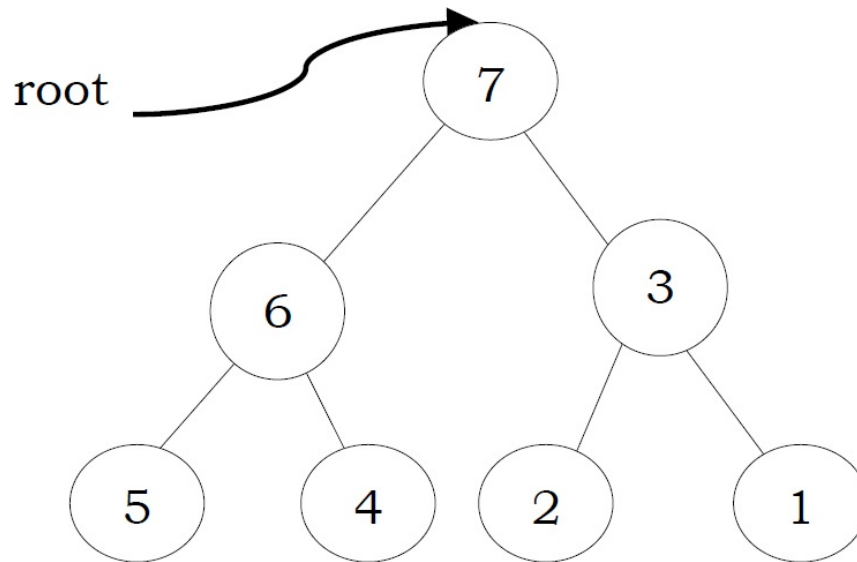
Problem-2 Is there a min-heap with seven distinct elements so that the preorder traversal of it gives the elements in sorted order?

Solution: Yes. For the tree below, preorder traversal produces ascending order.



Problem-3 Is there a max-heap with seven distinct elements so that the preorder traversal of it gives the elements in sorted order?

Solution: Yes. For the tree below, preorder traversal produces descending order.

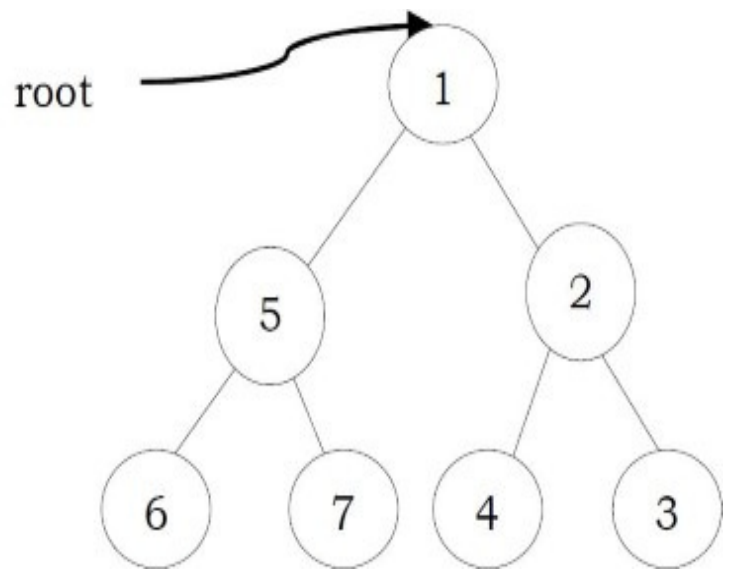
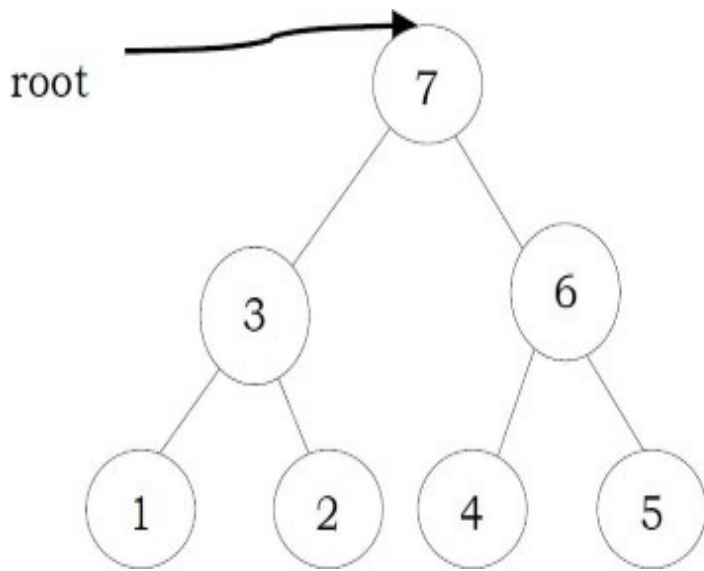


Problem-4 Is there a min-heap/max-heap with seven distinct elements so that the inorder traversal of it gives the elements in sorted order?

Solution: No. Since a heap must be either a min-heap or a max-heap, the root will hold the smallest element or the largest. An inorder traversal will visit the root of the tree as its second step, which is not the appropriate place if the tree's root contains the smallest or largest element.

Problem-5 Is there a min-heap/max-heap with seven distinct elements so that the postorder traversal of it gives the elements in sorted order?

Solution:



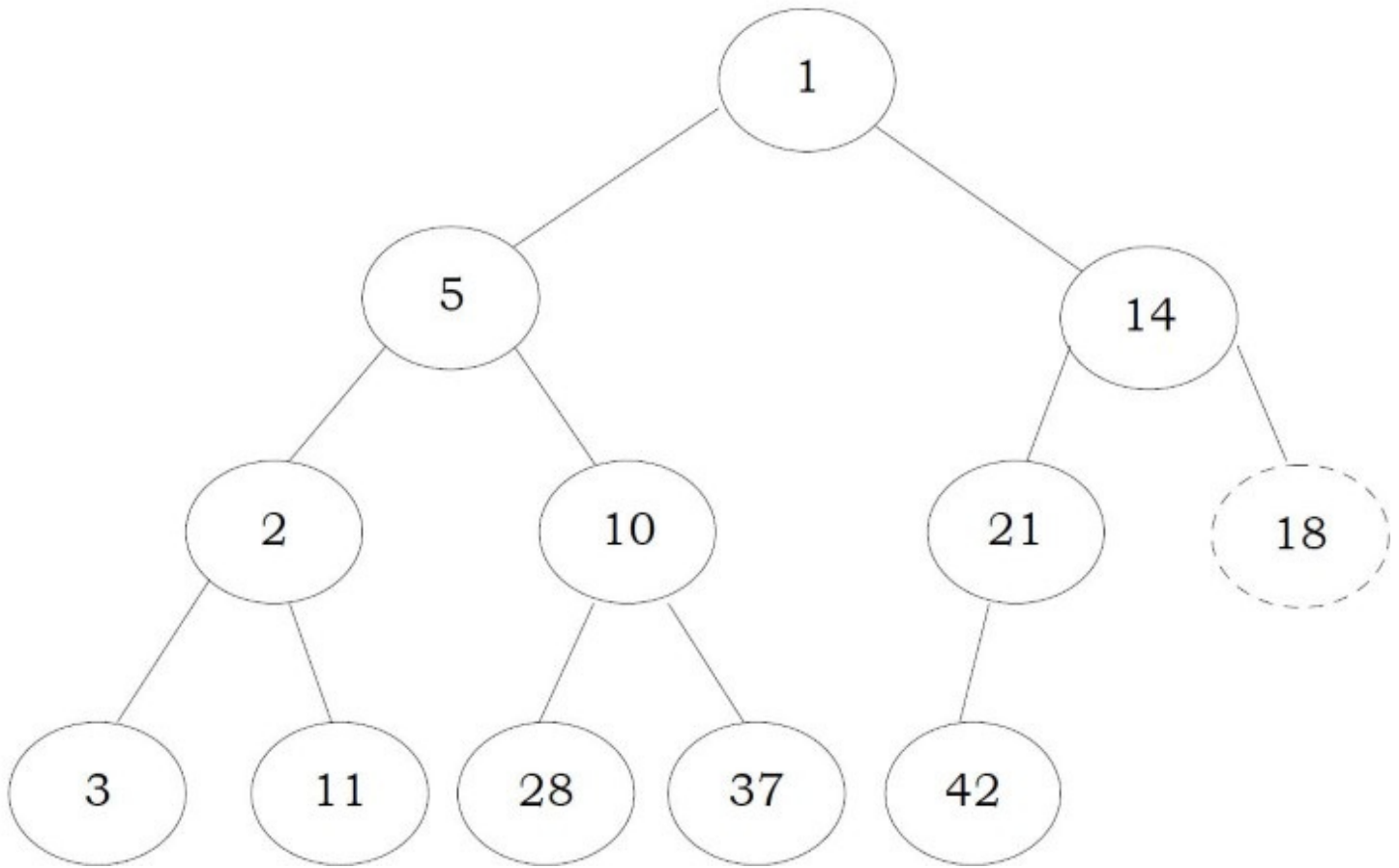
Yes, if the tree is a max-heap and we want descending order (below left), or if the tree is a min-heap and we want ascending order (below right).

Problem-6 Show that the height of a heap with n elements is $\log n$?

Solution: A heap is a complete binary tree. All the levels, except the lowest, are completely full. A heap has at least 2^h elements and at most elements $2^h \leq n \leq 2^{h+1} - 1$. This implies, $h \leq \log n \leq h + 1$. Since h is an integer, $h = \log n$.

Problem-7 Given a min-heap, give an algorithm for finding the maximum element.

Solution: For a given min heap, the maximum element will always be at leaf only. Now, the next question is how to find the leaf nodes in the tree.



If we carefully observe, the next node of the last element's parent is the first leaf node. Since the last element is always at the $h \rightarrow \text{count} - 1^{\text{th}}$ location, the next node of its parent (parent at location $\frac{h \rightarrow \text{count} - 1}{2}$) can be calculated as:

$$\frac{h \rightarrow \text{count} - 1}{2} + 1 \approx \frac{h \rightarrow \text{count} + 1}{2}$$

Now, the only step remaining is scanning the leaf nodes and finding the maximum among them.

```

int FindMaxInMinHeap(struct Heap *h) {
    int Max = -1;
    for(int i = (h->count+1)/2; i < h->count; i++)
        if(h->array[i] > Max)
            Max = h->array[i];
}
  
```

Time Complexity: $O\left(\frac{n}{2}\right) \approx O(n)$.

Problem-8 Give an algorithm for deleting an arbitrary element from min heap.

Solution: To delete an element, first we need to search for an element. Let us assume that we are using level order traversal for finding the element. After finding the element we need to follow the DeleteMin process.

Time Complexity = Time for finding the element + Time for deleting an element
 $= O(n) + O(\log n) \approx O(n)$. //Time for searching is dominated.

Problem-9 Give an algorithm for deleting the i^{th} indexed element in a given min-heap.

Solution:

```
int Delete(struct Heap *h, int i) {
    int key;
    if(n < i) {
        printf("Wrong position");
        return;
    }
    key = h->array[i];
    h->array[i] = h->array[h->count-1];
    h->count--;
    PercolateDown(h, i);
    return key;
}
```

Time Complexity = $O(\log n)$.

Problem-10 Prove that, for a complete binary tree of height h the sum of the height of all nodes is $O(n - h)$.

Solution: A complete binary tree has 2^i nodes on level i (Also, a node on level i has depth i and height $h - i$. Let us assume that S denotes the sum of the heights of all these nodes and S can be calculated as:

$$S = \sum_{i=0}^h 2^i (h - i)$$

$$S = h + 2(h - 1) + 4(h - 2) + \dots + 2^{h-1}(1)$$

Multiplying with 2 on both sides gives: $2S = 2h + 4(h - 1) + 8(h - 2) + \dots + 2^{h-1}(1)$

Now, subtract S from $2S$: $2S - S = -h + 2 + 4 + \dots + 2^h \Rightarrow S = (2^{h+1} - 1) - (h - 1)$

But, we already know that the total number of nodes n in a complete binary tree with height h is $n = 2^{h+1} - 1$. This gives us: $h = \log(n + 1)$.

Finally, replacing $2^{h+1} - 1$ with n , gives: $S = n - (h - 1) = O(n - \log n) = O(n - h)$.

Problem-11 Give an algorithm to find all elements less than some value of k in a binary heap.

Solution: Start from the root of the heap. If the value of the root is smaller than k then print its value and call recursively once for its left child and once for its right child. If the value of a node is greater or equal than k then the function stops without printing that value.

The complexity of this algorithm is $O(n)$, where n is the total number of nodes in the heap. This bound takes place in the worst case, where the value of every node in the heap will be smaller than k , so the function has to call each node of the heap.

Problem-12 Give an algorithm for merging two binary max-heaps. Let us assume that the size of the first heap is $m + n$ and the size of the second heap is n .

Solution: One simple way of solving this problem is:

- Assume that the elements of the first array (with size $m + n$) are at the beginning. That means, first m cells are filled and remaining n cells are empty.
- Without changing the first heap, just append the second heap and heapify the array.
- Since the total number of elements in the new array is $m + n$, each heapify operation takes $O(\log(m + n))$.

The complexity of this algorithm is : $O((m + n)\log(m + n))$.

Problem-13 Can we improve the complexity of [Problem-12](#)?

Solution: Instead of heapifying all the elements of the $m + n$ array, we can use the technique of “building heap with an array of elements (heapifying array)”. We can start with non-leaf nodes and heapify them. The algorithm can be given as:

- Assume that the elements of the first array (with size $m + n$) are at the beginning. That means, the first m cells are filled and the remaining n cells are empty.
- Without changing the first heap, just append the second heap.
- Now, find the first non-leaf node and start heapifying from that element.

In the theory section, we have already seen that building a heap with n elements takes $O(n)$ complexity. The complexity of merging with this technique is: $O(m + n)$.

Problem-14 Is there an efficient algorithm for merging 2 max-heaps (stored as an array)? Assume both arrays have n elements.

Solution: The alternative solution for this problem depends on what type of heap it is. If it's a standard heap where every node has up to two children and which gets filled up so that the leaves are on a maximum of two different rows, we cannot get better than $O(n)$ for the merge.

There is an $O(\log m \times \log n)$ algorithm for merging two binary heaps with sizes m and n . For $m = n$, this algorithm takes $O(\log^2 n)$ time complexity. We will be skipping it due to its difficulty and scope.

For better merging performance, we can use another variant of binary heap like a *Fibonacci-Heap* which can merge in $O(1)$ on average (amortized).

Problem-15 Give an algorithm for finding the k^{th} smallest element in min-heap.

Solution: One simple solution to this problem is: perform deletion k times from min-heap.

```
return PQ.Min();  
//Just delete first k-1 elements and return the k-th element.  
for(int i=0;i<k-1;i++)  
    DeleteMin(h);  
return DeleteMin(h);  
}
```

Time Complexity: $O(k \log n)$. Since we are performing deletion operation k times and each deletion takes $O(\log n)$.

Problem-16 For [Problem-15](#), can we improve the time complexity?

Solution: Assume that the original min-heap is called H_{Orig} and the auxiliary min-heap is named H_{Aux} . Initially, the element at the top of H_{Orig} , the minimum one, is inserted into H_{Aux} . Here we don't do the operation of `DeleteMin` with H_{Orig} .

```

Heap HOrig;
Heap HAux;
int FindKthLargestEle( int k ) {
    int heapElement; // Assuming heap data is of integers
    int count=1;
    HAux.Insert(HOrig.Min());
    while( true ) {
        //return the minimum element and delete it from the HA heap
        heapElement = HAux.DeleteMin();
        if(++count == k ) {
            return heapElement;
        }
        else { //insert the left and right children in HO into the HA
            HAux.Insert(heapElement.LeftChild());
            HAux.Insert(heapElement.RightChild());
        }
    }
}

```

Every while-loop iteration gives the k^{th} smallest element and we need k loops to get the k^{th} smallest elements. Because the size of the auxiliary heap is always less than k , every while-loop iteration the size of the auxiliary heap increases by one, and the original heap H_{Orig} has no operation during the finding, the running time is $O(k \log k)$.

Note: The above algorithm is useful if the k value is too small compared to n . If the k value is approximately equal to n , then we can simply sort the array (let's say, using *counting* sort or any other linear sorting algorithm) and return k^{th} smallest element from the sorted array. This gives $O(n)$ solution.

Problem-17 Find k max elements from max heap.

Solution: One simple solution to this problem is: build max-heap and perform deletion k times.

$$T(n) = \text{DeleteMin from heap } k \text{ times} = \Theta(k \log n).$$

Problem-18 For [Problem-17](#), is there any alternative solution?

Solution: We can use the [Problem-16](#) solution. At the end, the auxiliary heap contains the k -largest elements. Without deleting the elements we should keep on adding elements to $HAux$.

Problem-19 How do we implement stack using heap?

Solution: To implement a stack using a priority queue PQ (using min heap), let us assume that we are using one extra integer variable c . Also, assume that c is initialized equal to any known value (e.g., 0). The implementation of the stack ADT is given below. Here c is used as the priority while inserting/deleting the elements from PQ.

```
void Push(int element) {
    PQ.Insert(c, element);
    c--;
}

int Pop() {
    return PQ.DeleteMin();
}

int Top() {
    return PQ.Min();
}

int Size() {
    return PQ.Size();
}

int IsEmpty() {
    return PQ.IsEmpty();
}
```

We could also increment c back when popping.

Observation: We could use the negative of the current system time instead of c (to avoid overflow). The implementation based on this can be given as:

```
void Push(int element) {
    PQ.insert(-gettime(), element);
}
```

Problem-20 How do we implement Queue using heap?

Solution: To implement a queue using a priority queue PQ (using min heap), as similar to stacks simulation, let us assume that we are using one extra integer variable, c . Also, assume that c is initialized equal to any known value (e.g., 0). The implementation of the queue ADT is given below. Here the c is used as the priority while inserting/deleting the elements from PQ.


```

void Push(int element) {
    PQ.Insert(c, element);
    c++;
}

int Pop() {
    return PQ.DeleteMin();
}

int Top() {
    return PQ.Min();
}

int Size() {
    return PQ.Size();
}

int IsEmpty() {
    return PQ.IsEmpty();
}

```

Note: We could also decrement c when popping.

Observation: We could use just the negative of the current system time instead of c (to avoid overflow). The implementation based on this can be given as:

```

void Push(int element) {
    PQ.insert(gettime(),element);
}

```

Note: The only change is that we need to take a positive c value instead of negative.

Problem-21 Given a big file containing billions of numbers, how can you find the 10 maximum numbers from that file?

Solution: Always remember that when you need to find max n elements, the best data structure to use is priority queues.

One solution for this problem is to divide the data in sets of 1000 elements (let's say 1000) and make a heap of them, and then take 10 elements from each heap one by one. Finally heap sort all the sets of 10 elements and take the top 10 among those. But the problem in this approach is where to store 10 elements from each heap. That may require a large amount of memory as we

have billions of numbers.

Reusing the top 10 elements (from the earlier heap) in subsequent elements can solve this problem. That means take the first block of 1000 elements and subsequent blocks of 990 elements each. Initially, Heapsort the first set of 1000 numbers, take max 10 elements, and mix them with 990 elements of the 2^{nd} set. Again, Heapsort these 1000 numbers (10 from the first set and 990 from the 2^{nd} set), take 10 max elements, and mix them with 990 elements of the 3^{rd} set. Repeat till the last set of 990 (or less) elements and take max 10 elements from the final heap. These 10 elements will be your answer.

Time Complexity: $O(n) = n/1000 \times (\text{complexity of Heapsort 1000 elements})$ Since complexity of heap sorting 1000 elements will be a constant so the $O(n) = n$ i.e. linear complexity.

Problem-22 Merge k sorted lists with total of n elements: We are given k sorted lists with total n inputs in all the lists. Give an algorithm to merge them into one single sorted list.

Solution: Since there are k equal size lists with a total of n elements, the size of each list is $\frac{n}{k}$. One simple way of solving this problem is:

- Take the first list and merge it with the second list. Since the size of each list is $\frac{n}{k}$, this step produces a sorted list with size $\frac{2n}{k}$. This is similar to merge sort logic. The time complexity of this step is: $\frac{2n}{k}$. This is because we need to scan all the elements of both the lists.
- Then, merge the second list output with the third list. As a result, this step produces a sorted list with size $\frac{3n}{k}$. The time complexity of this step is: $\frac{3n}{k}$. This is because we need to scan all the elements of both lists (one with size $\frac{2n}{k}$ and the other with size $\frac{n}{k}$).
- Continue this process until all the lists are merged to one list.

Total time complexity:
$$= \frac{2n}{k} + \frac{3n}{k} + \frac{4n}{k} + \dots + \frac{kn}{k} = \sum_{i=2}^n \frac{i n}{k} = \frac{n}{k} \sum_{i=2}^n i \approx \frac{n(k^2)}{k} \approx O(nk)$$

Space Complexity: $O(1)$.

Problem-23 For [Problem-22](#), can we improve the time complexity?

Solution:

- 1 Divide the lists into pairs and merge them. That means, first take two lists at a time and merge them so that the total elements parsed for all lists is $O(n)$. This operation gives $k/2$ lists.
- 2 Repeat step-1 until the number of lists becomes one.

Time complexity: Step-1 executes $\log k$ times and each operation parses all n elements in all the lists for making $k/2$ lists. For example, if we have 8 lists, then the first pass would make 4 lists by

parsing all n elements. The second pass would make 2 lists by again parsing n elements and the third pass would give 1 list by again parsing n elements. As a result the total time complexity is $O(n \log n)$.

Space Complexity: $O(n)$.

Problem-24 For [Problem-23](#), can we improve the space complexity?

Solution: Let us use heaps for reducing the space complexity.

1. Build the max-heap with all the first elements from each list in $O(k)$.
2. In each step, extract the maximum element of the heap and add it at the end of the output.
3. Add the next element from the list of the one extracted. That means we need to select the next element of the list which contains the extracted element of the previous step.
4. Repeat step-2 and step-3 until all the elements are completed from all the lists.

Time Complexity = $O(n \log k)$. At a time we have k elements max-heap and for all n elements we have to read just the heap in $\log k$ time, so total time = $O(n \log k)$.

Space Complexity: $O(k)$ [for Max-heap].

Problem-25 Given 2 arrays A and B each with n elements. Give an algorithm for finding largest n pairs $(A[i], B[j])$.

Solution:

Algorithm:

- Heapify A and B . This step takes $O(2n) \approx O(n)$.
- Then keep on deleting the elements from both the heaps. Each step takes $O(2 \log n) \approx O(\log n)$.

Total Time complexity: $O(n \log n)$.

Problem-26 Min-Max heap: Give an algorithm that supports min and max in $O(1)$ time, insert, delete min, and delete max in $O(\log n)$ time. That means, design a data structure which supports the following operations:

Operation	Complexity
Init	$O(n)$
Insert	$O(\log n)$
FindMin	$O(1)$
FindMax	$O(1)$
Delete Min	$O(\log n)$

Delete Max

 $O(\log r_i)$

Solution: This problem can be solved using two heaps. Let us say two heaps are: Minimum-Heap $H_{\min}$ and Maximum-Heap $H_{\max}$. Also, assume that elements in both the arrays have mutual pointers. That means, an element in $H_{\min}$ will have a pointer to the same element in $H_{\max}$ and an element in $H_{\max}$ will have a pointer to the same element in $H_{\min}$.

Init	Build $H_{\min}$ in $O(n)$ and $H_{\max}$ in $O(n)$
Insert(x)	Insert x to $H_{\min}$ in $O(\log n)$. Insert x to $H_{\max}$ in $O(\log n)$. Update the pointers in $O(1)$
FindMin()	Return root($H_{\min}$) in $O(1)$
FindMax	Return root($H_{\max}$) in $O(1)$
Delete Min	Delete the minimum from $H_{\min}$ in $O(\log n)$. Delete the same element from $H_{\max}$ by using the mutual pointer in $O(\log n)$
DeleteMax	Delete the maximum from $H_{\max}$ in $O(\log n)$. Delete the same element from $H_{\min}$ by using the mutual pointer in $O(\log n)$

Problem-27 Dynamic median finding. Design a heap data structure that supports finding the median.

Solution: In a set of n elements, median is the middle element, such that the number of elements lesser than the median is equal to the number of elements larger than the median. If n is odd, we can find the median by sorting the set and taking the middle element. If n is even, the median is usually defined as the average of the two middle elements. This algorithm works even when some of the elements in the list are equal. For example, the median of the multiset $\{1, 1, 2, 3, 5\}$ is 2, and the median of the multiset $\{1, 1, 2, 3, 5, 8\}$ is 2.5.

“Median heaps” are the variant of heaps that give access to the median element. A median heap can be implemented using two heaps, each containing half the elements. One is a max-heap, containing the smallest elements; the other is a min-heap, containing the largest elements. The size of the max-heap may be equal to the size of the min-heap, if the total number of elements is even. In this case, the median is the average of the maximum element of the max-heap and the minimum element of the min-heap. If there is an odd number of elements, the max-heap will contain one more element than the min-heap. The median in this case is simply the maximum element of the max-heap.

Problem-28 **Maximum sum in sliding window:** Given array $A[]$ with sliding window of size w which is moving from the very left of the array to the very right. Assume that we can only see the w numbers in the window. Each time the sliding window moves rightwards by

one position. For example: The array is [1 3 -1 -3 5 3 6 7], and w is 3.

Window position	Max
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

Input: A long array A[], and a window width w. **Output:** An array B[], B[i] is the maximum value of from A[i] to A[i+w-1]

Requirement: Find a good optimal way to get B[i]

Solution: Brute force solution is, every time the window is moved we can search for a total of w elements in the window.

Time complexity: $O(nw)$.

Problem-29 For [Problem-28](#), can we reduce the complexity?

Solution: Yes, we can use heap data structure. This reduces the time complexity to $O(n\log w)$. Insert operation takes $O(\log w)$ time, where w is the size of the heap. However, getting the maximum value is cheap; it merely takes constant time as the maximum value is always kept in the root (head) of the heap. As the window slides to the right, some elements in the heap might not be valid anymore (range is outside of the current window). How should we remove them? We would need to be somewhat careful here. Since we only remove elements that are out of the window's range, we would need to keep track of the elements' indices too.

Problem-30 For [Problem-28](#), can we further reduce the complexity?

Solution: Yes, The double-ended queue is the perfect data structure for this problem. It supports insertion/deletion from the front and back. The trick is to find a way such that the largest element in the window would always appear in the front of the queue. How would you maintain this requirement as you push and pop elements in and out of the queue?

Besides, you will notice that there are some redundant elements in the queue that we shouldn't even consider. For example, if the current queue has the elements: [10 5 3], and a new element in the window has the element 11. Now, we could have emptied the queue without considering elements 10, 5, and 3, and insert only element 11 into the queue.